Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

A Personal Knowledge Assistant: Design and Evaluation of a Retrieval-Augmented Generation System for Document-Grounded Question Answering

By
Patrick Clerkin

April 26, 2026

B.Sc. (Hons) in Software Development

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Abstract

Retrieval-Augmented Generation (RAG) addresses two persistent weaknesses of large language models for document-grounded question answering: hallucination, and the inability to access knowledge outside the training corpus. This dissertation presents the Personal Knowledge Assistant, a complete end-to-end RAG system that ingests a user's own documents in four formats, indexes them in a hybrid lexical-and-semantic retrieval system, and answers natural-language questions through an eleven-stage pipeline incorporating semantic caching, conversational query rewriting, Hypothetical Document Embeddings, sentence-level grounding, adaptive re-retrieval, and post-generation fact verification. The system is evaluated through two parallel layers: classical information-retrieval metrics for fast regression testing and a from-scratch RAGAS-style answer evaluator measuring faithfulness, answer relevancy, and context precision against labelled question sets. Across two structurally distinct corpora — twelve seminal machine-learning papers and a third-year machine-learning course's lecture notes — and twenty hand-labelled queries, the system achieves a mean faithfulness of 0.934 and a composite RAGAS score of 0.788, with 313 atomic claims extracted and the great majority judged as supported by retrieved context. The architecture comprises six layers separated by clean abstraction boundaries, applies eleven classical design patterns, and is verified by an automated test suite of 522 tests across 29 modules. The contribution is not novelty in any single technique, all of which are drawn from established literature, but in their disciplined integration into a coherent, evaluable system, with the evaluation framework instrumented to make the system's behaviour examinable, subject to the validity threats catalogued in Chapter 5.

Contents

Abstract **i**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Approach: Retrieval-Augmented Generation . . . 2
1.3 The Personal Knowledge Assistant . . . 2
1.4 Project Objectives . . . 3
1.5 Scope and Constraints . . . 4
1.6 Contribution . . . 4
1.7 Dissertation Structure . . . 5

2 Methodology **6**
2.1 Approach . . . 6
2.2 Project Planning and Supervision . . . 6
2.3 Iterative-Incremental Development . . . 7
2.4 Test-Driven Validation . . . 8
2.5 Technology Selection . . . 8
2.6 Empirical Problem-Solving . . . 10
2.7 Scope Evolution . . . 11

3 Technology Review **13**
3.1 Introduction and Scope . . . 13
3.2 Review Methodology . . . 13
3.3 Lexical Information Retrieval . . . 14
3.4 Dense Sentence Embeddings . . . 14
3.5 Approximate Nearest-Neighbour Search . . . 15
3.6 Hybrid Retrieval and Reciprocal Rank Fusion . . . 16
3.7 Cross-Encoder Reranking . . . 17
3.8 Document Chunking Strategies . . . 17
3.9 Query Expansion and Hypothetical Document Embeddings . . . 18
3.10 Retrieval-Augmented Generation . . . 19

3.11 Evaluation of Retrieval-Augmented Systems . . . 20
3.12 Knowledge Graphs from Unstructured Text . . . 21
3.13 Synthesis . . . 21

4 System Design 23
4.1 Architectural Overview . . . 23
4.2 Ingestion Pipeline . . . 23
4.3 Storage Layer . . . 26
4.4 Retrieval Layer . . . 26
4.5 The RAG Pipeline . . . 28
4.6 Intelligence Layer . . . 29
4.7 Two Independent Evaluation Layers . . . 30
4.8 Design Pattern Catalogue . . . 31
4.9 Interface Layer . . . 32
4.10 Design Tensions . . . 33

5 System Evaluation 34
5.1 Evaluation Methodology . . . 34
5.2 Headline Results . . . 35
5.3 Baseline Comparison . . . 36
5.4 Runtime Performance . . . 38
5.5 Qualitative Analysis: Best and Worst Cases . . . 39
5.6 Mapping Results to Project Objectives . . . 40
5.7 Human Calibration of the LLM Judge . . . 42
5.8 Threats to Validity . . . 43
5.9 Limitations . . . 44

6 Conclusion 47
6.1 Outcomes . . . 47
6.2 Principal Findings . . . 48
6.3 Future Work . . . 48
6.4 Critical Reflection . . . 49
6.5 Closing Reflection . . . 50

A Project Repository 55

List of Figures

4.1 Six-layer architecture. Each layer depends on the layer below. Cross-cutting concerns (logging, configuration) span the stack. 24
4.2 Hybrid retrieval. Two parallel search paths fuse via RRF; the fused list is re-scored by a cross-encoder. 27
4.3 Eleven-stage RAG pipeline. Solid arrows: main path. Dashed arrows: the adaptive-retry loop, which re-runs retrieval with relaxed parameters when the grounding score is below threshold; the highest-confidence attempt across retries is kept. 28

5.1 Per-metric performance across the two evaluation corpora. Faithfulness varied little between corpora; the other three metrics varied more with document structure. 36

List of Tables

2.1 Principal project phases. Each phase delivered concrete working capability before the next began. 7
2.2 Distribution of the 522 unit and integration tests across functional areas. Counts derived directly from the test source files. 9

3.1 Comparative trade-offs of approximate nearest-neighbour approaches. Ranges reflect cross-study variation in hyperparameters and hardware [1, 2, 3]. 16

4.1 The six implemented chunking strategies. `ChunkManager.recommend_strategy()` chooses among them based on document length and section count. . 25
4.2 Design patterns used in the system, with concrete classes from the source repository. 32

5.1 Answer-quality metrics across the two evaluation corpora. NLP-Papers: $n = 12$; ML-Notes: $n = 8$; combined $n = 20$. 35
5.2 Naive BM25-only baseline against the full pipeline on the NLP-Papers corpus (n=12). The same generator, knowledge base, chunker, prompt template, and test set are used for both runs; only the retrieval-and-grounding apparatus differs. 37
5.3 End-to-end latency for representative operations against an active knowledge base of 3,330 chunks. Cold values include one-time model loading; warm values are steady-state. 39
5.4 Highest- and lowest-scoring queries per corpus, with the component metric most responsible for the score. 39

Chapter 1

Introduction

1.1 Motivation

Large language models have become a widely used interface for accessing information, but their reliability for knowledge-intensive tasks is constrained by two well-documented weaknesses. The first is hallucination: the tendency to generate fluent, plausible-sounding text that is factually incorrect or that fabricates citations and details that have no basis in any source material [4]. The second is knowledge isolation: parametric language models are trained on a fixed corpus, so any document the user owns but the model has not seen — a personal notebook, a downloaded research paper, a company report, a course handout — is invisible to the model regardless of its relevance to the user's question. The two weaknesses compound. A user who asks an unaugmented language model a question about a document it has not been trained on receives either a refusal, a confident-sounding fabrication, or, most insidiously, an answer that mixes generic knowledge from the training corpus with confabulated specifics that match the user's question superficially without being grounded in any actual source.

The practical consequence is a category of work for which language models are nearly but not quite useful. A student studying an academic paper, a researcher cross-referencing a small set of references, an engineer consulting product documentation, or anyone returning to their own notes weeks after writing them, would benefit from being able to question their own document collection conversationally and receive cited, verifiable answers. The naive approach — copy-pasting the document into the chat window — works only for short documents and provides no facility for cross-document synthesis, search across a collection, or multi-turn follow-up. The question motivating this dissertation is therefore practical: how can a single user, with a personal document collection and a commodity machine, build a system that answers questions about their own documents reliably, with

citations, and without inventing sources that do not exist?

1.2 Approach: Retrieval-Augmented Generation

The architectural answer adopted by this work is Retrieval-Augmented Generation (RAG) [5]. RAG decouples knowledge storage from language generation: the user's documents are indexed into a retrieval system, and at query time, the relevant passages are retrieved and supplied to a language model alongside the question. The language model's role narrows from recalling facts to synthesising a coherent answer from explicitly retrieved evidence, which can then be cited. The architecture addresses both motivating weaknesses in a single move: knowledge is no longer isolated, because new documents can be ingested without retraining; and hallucination is reduced, because the generator is conditioned on retrieved context that the answer can be grounded in. Subsequent literature has established RAG as a deployable architecture for knowledge-intensive tasks [6], and a growing body of empirical work characterises its strengths, robustness boundaries, and evaluation methodology [7, 8].

The integration of established RAG techniques into a coherent, end-to-end personal system — spanning ingestion, retrieval, generation, and evaluation — is the engineering contribution of this dissertation. The system is called the Personal Knowledge Assistant.

1.3 The Personal Knowledge Assistant

The Personal Knowledge Assistant is a Python application that ingests a user's documents (PDF, DOCX, Markdown, plain text), indexes them in a hybrid lexical-and-semantic retrieval system, and answers natural-language questions by retrieving relevant passages and generating cited answers via a language model. It runs locally as a Flask web application; the same backend is also accessible through a command-line interface and as a Python library. Ingestion is incremental: the system tracks document hashes and re-ingests only changed files. Retrieval is hybrid: it combines lexical BM25 search [9] with dense semantic search over sentence-transformer embeddings [10] fused via Reciprocal Rank Fusion [11], with cross-encoder reranking [12] and three configurable query-expansion strategies. Generation is grounded: the system uses sentence-level grounding scoring and adaptive re-retrieval to detect and recover from low-confidence answers. Evaluation is two-layered: classical information-retrieval metrics [13] support fast regression testing during development, and a from-scratch RAGAS-style answer evaluator [8] measures faithfulness, answer relevancy, and context precision against labelled question

sets. Beyond question-answering, the system also constructs a knowledge graph of co-occurring entities from ingested documents, detects factual conflicts between source documents, and generates study paths, quizzes, and document summaries.

1.4 Project Objectives

The system was developed against six concrete objectives. These are the objectives stated here and evaluated against in Chapter 5; the relationship between this revised set and the original October 2025 proposal is documented in Section 2.7.

O1: Robust multi-format ingestion.
The system shall ingest documents in at least four formats (PDF, DOCX, Markdown, plain text) through a uniform interface, detect changes via cryptographic hashing, and short-circuit re-ingestion of unchanged files.

O2: Multiple chunking strategies under a uniform interface.
The system shall implement at least six distinct chunking strategies spanning lexical (fixed-size, sentence-boundary), semantic (embedding-similarity), density-based (HDBSCAN), topic-modelling (BERTopic), and structural (recursive-hierarchical) approaches, selectable through a single configuration interface, supporting empirical comparison on a common corpus.

O3: Hybrid retrieval with reranking and query expansion.
The system shall combine BM25 lexical retrieval with FAISS dense semantic retrieval, fuse the two using Reciprocal Rank Fusion, apply cross-encoder reranking to the fused candidate pool, and offer at least three query-expansion strategies including Hypothetical Document Embeddings (HyDE).

O4: Full Advanced-RAG pipeline with grounding and recovery.
The system shall implement an end-to-end retrieval-then-generation pipeline including semantic caching, conversational query rewriting, HyDE expansion, sentence-level grounding scoring, adaptive re-retrieval on low-confidence answers, and post-generation fact verification, all coordinated through a single orchestrator with clean abstraction boundaries between components.

O5: Two-layer evaluation framework.
The system shall expose two independent evaluation layers: a classical information-retrieval metric module (Precision@K, MRR, nDCG@K) for fast regression checks, and a from-scratch RAGAS-style answer evaluator computing faithfulness, answer relevancy, and context precision against labelled question sets.

O6: Three interfaces and a verified test suite.
The system shall be accessible through a command-line interface, a Python API, and a Flask web application sharing a common backend, with all behaviour verified by an automated test suite of at least five hundred unit and integration tests across at least twenty-five modules.

These objectives are concrete and binary: each is either met or not, and Chapter 5 reports the outcome against each.

1.5 Scope and Constraints

The system targets the personal-corpus scale: knowledge bases of roughly thousands of documents and tens of thousands of chunks, on single-user hardware, with the user's own document collection as the exclusive knowledge source. Three constraints follow from this scoping.

The system is not a search engine for the open web; it has no internet-scale crawl, no advertiser-incentive ranking model, and no mechanism for evaluating sources outside the documents the user has chosen to ingest. The system is not a multi-tenant service; there is no authentication, authorisation, or per-user data isolation, on the assumption that a single user owns and operates the deployment. The system is not a replacement for the underlying language model on general-knowledge questions; for queries the user's documents do not address, the system declines to answer rather than falling back to the language model's parametric knowledge, a behaviour examined empirically in Section 5.5.

These exclusions are deliberate: the system optimises for depth in a narrow use case (reliable question-answering against the user's own documents) rather than for breadth across multiple use cases.

1.6 Contribution

The work contributes a complete, evaluated, end-to-end personal knowledge assistant system, with three aspects worth highlighting in particular.

First, the system integrates a substantial number of established retrieval and generation techniques into a single coherent architecture: hybrid retrieval, reranking, query expansion including HyDE, semantic caching, conversational memory, sentence-level grounding, fact verification, and adaptive re-retrieval all coexist and cooperate through clean abstraction boundaries (Chapter 4). The integration itself is the contribution; the constituent techniques are not novel, but their composition into a working, testable, evaluable system is non-trivial.

Second, the evaluation is rigorous in a way that comparable undergraduate projects often are not. The from-scratch RAGAS-style answer evaluator (Section 4.7) reports faithfulness, answer relevancy, and context precision against twenty hand-labelled queries across two structurally distinct corpora; the results are analysed both at the aggregate metric level and at the per-question level, with two failure modes (a verbosity failure and a graceful-degradation success) examined in detail (Section 5.5). The honest acknowledgement of limitations — same-family judge bias, single-run nondeterminism, test-set size — is part of the contribution: it makes the conclusions trustworthy rather than overstated.

Third, the system is verified by an automated test suite of 522 tests across 29 modules, exceeding the originally stated objective threshold and providing the regression discipline that allowed the architecture to be refined incrementally without accumulated drift.

1.7 Dissertation Structure

The remainder of this dissertation is structured as follows. Chapter 2 describes the software-development methodology under which the system was built, including the iterative-incremental approach, the testing discipline, the technology-selection rationale, and the explicit documentation of how the project's scope evolved relative to its October 2025 proposal. Chapter 3 surveys the technical literature on lexical and dense retrieval, approximate nearest-neighbour search, hybrid retrieval and reranking, chunking strategies, query expansion, the RAG paradigm, and evaluation methodology, identifying the design choices each section motivates. Chapter 4 presents the system's architecture across six layers (ingestion, storage, retrieval, RAG pipeline, intelligence, interfaces), with concrete class and method references that map directly to the source repository, and concludes with an honest acknowledgement of the architecture's design tensions. Chapter 5 reports the outcome of evaluating the system against the six objectives, presents quantitative answer-quality metrics across two evaluation corpora, compares the full pipeline against a stripped-down BM25 baseline, reports a manual human-rater calibration of the LLM judge, analyses the two qualitatively distinct failure modes, includes runtime performance benchmarks, and catalogues the validity threats and limitations that bound the conclusions. Chapter 6 summarises the principal findings against each objective and sets out the broader research and engineering directions that follow from this work.

Chapter 2

Methodology

2.1 Approach

This dissertation reports a software engineering effort rather than a research investigation, and the methodology reflects that distinction. The objective was to build a working system end-to-end, not to test a single hypothesis under controlled conditions. Accordingly the methodology is iterative and incremental, drawing on the principles articulated in the Agile Manifesto [14]: prioritising working software over comprehensive documentation, responding to change rather than adhering rigidly to a plan, and developing in short cycles that produce demonstrable value. There was no separate research phase running in advance of implementation; instead, literature review, design, and implementation proceeded in parallel, with each informing the others as the system grew. Where research questions arose — for example, how to combine BM25 and FAISS scores, or how to detect hallucination at generation time — they were treated as concrete engineering problems with established solutions in the literature (Chapter 3) rather than as open empirical questions. This positioning is important because it shapes what counts as methodological rigour: the system is evaluated against the objectives it was built to satisfy (Chapter 5), rather than against a controlled experimental hypothesis.

2.2 Project Planning and Supervision

The project followed the structure mandated by the Department of Computer Science and Applied Physics for the BSc Applied Project and Minor Dissertation module. A formal proposal was submitted in late October 2025 setting out objectives, technology choices, schedule, and milestones; that proposal was reviewed and signed off by the project supervisor before implementation began. Supervisor meetings were held periodically through the development period (October 2025

Table 2.1: Principal project phases. Each phase delivered concrete working capability before the next began.

Phase	Capability delivered
1	Proposal accepted; environment and project structure in place
2	Document ingestion and basic question-answering operational
3	Semester 1 prototype demonstration
4	Web chat interface, conversation memory, streaming responses
5	Hybrid retrieval, cross-encoder reranking, query expansion
6	Sentence-level grounding, fact verification, evaluation suite
7	Final evaluation runs and dissertation drafting

to April 2026) to discuss progress, surface blocking issues, and calibrate scope. The phase plan, summarised in Table 2.1, was completed in sequence; the most consequential planning decision, the deliberate revision of project scope partway through development, is documented explicitly in Section 2.7.

2.3 Iterative-Incremental Development

Implementation proceeded in feature-sized increments. Each increment followed a consistent rhythm: identify the feature against the proposal, locate the relevant technique in the literature, design the abstraction (an interface, a class, a small module), implement against that abstraction, write tests covering both the happy path and the key edge cases, and integrate into the broader system through the existing layered architecture (Section 4.1). The discipline of designing the abstraction before writing the implementation is responsible for the testability of the resulting system: components such as `LLMProvider`, `HybridSearcher`, and `GroundingScorer` are straightforward to mock or substitute in tests precisely because their interfaces were articulated before their implementations existed.

The git commit history serves as the audit trail of this process. Commit messages are structured to identify the feature or fix being introduced, and commits are grouped logically so that a reader (or the author returning to the work later)

can reconstruct the order in which capabilities were added. Because the system was developed solo, no branching or pull-request workflow was strictly required. The timestamps in the commit history align with the milestones described in Section 2.2, providing external corroboration of the project schedule.

2.4 Test-Driven Validation

Validation is implemented through an automated test suite of 522 unit and integration tests across 29 test modules (Section 4.7). The suite was not produced as a final deliverable; it grew alongside the codebase, with tests written either immediately before or immediately after the corresponding production code. This discipline served three concrete purposes during development. First, it enabled rapid regression checking: changing the grounding-score formula, replacing the chunk-similarity metric, or refactoring the storage layer could each be done with confidence that any inadvertent breakage would surface within seconds. Second, it captured behavioural specifications in executable form: a test such as `test_pipeline_empty_kb`, which verifies that the RAG pipeline returns a graceful response when the knowledge base is empty, documents the expected behaviour more precisely than prose documentation would. Third, it allowed mocked LLM providers and synthetic corpora to substitute for the real Claude API during testing, so that the full suite runs in seconds rather than the minutes a real LLM evaluation would require.

The test suite covers all six chunkers, both storage backends, hybrid retrieval, the eleven-stage RAG pipeline including the empty-knowledge-base edge case, the grounding scorer, the fact verifier, the answer evaluator, the knowledge graph, the conflict detector, the study-path generator, the quiz generator, the summariser, and the web API endpoints. The categorised distribution of tests across functional areas is shown in Table 2.2. Coverage is breadth-oriented rather than line-percentage-oriented: the test suite catches the classes of failure most likely to occur, rather than chasing a particular coverage figure that may or may not correspond to genuine robustness.

2.5 Technology Selection

Technology choices were driven by three constraints: rapid prototyping, alignment with the techniques surveyed in Chapter 3, and the engineering economics of a single developer working on a six-month timeline. The technology stack described below produces a system of roughly 13,000 lines of Python across 73 source modules (excluding the test suite); the choices below were made with a working knowledge

Table 2.2: Distribution of the 522 unit and integration tests across functional areas. Counts derived directly from the test source files.

Functional area	**Tests**
Ingestion (parsers, chunkers, embeddings, NER, registry)	112
Storage and retrieval (vector store, BM25, hybrid, reranker)	76
RAG pipeline (cache, memory, grounding, fact verifier, etc.)	99
Intelligence layer (graph, conflicts, study, quiz, summary)	68
Evaluation (IR metrics, RAGAS, endpoints)	64
Web API and CLI (endpoints, annotations, history, CLI)	103
Total	522

of that scale and of the maintenance load it implied.

Python was selected as the implementation language because it is the lingua franca of the machine-learning ecosystem: every established library for embedding models, vector search, NER, topic modelling, and graph analysis exposes a Python interface. The alternative of a JVM-based language would have required wrapping foreign libraries through bindings of variable quality. **Flask** was chosen over FastAPI for the web layer despite the proposal specifying the latter; Flask's longer history and simpler request model reduced ramp-up time for the relatively conventional REST API the system needed, and the App-Factory plus Blueprint pattern (Section 4.9) provided sufficient structure without introducing the additional concepts FastAPI's dependency-injection model brings.

FAISS was chosen as the vector store after considering Qdrant, Chroma, and Weaviate. For the personal-corpus scale at which the system operates, the operational simplicity of an in-process library that persists to disk as ordinary files outweighed the multi-tenancy and distribution capabilities of full-database alternatives. Faiss's `IndexFlatIP` delivers exact cosine similarity at the corpus sizes characteristic of the use case (Section 4.3). **rank-bm25** provided the lexical complement to FAISS without adding infrastructure. **sentence-transformers**, specifically `all-MiniLM-L6-v2`, was chosen for embeddings on the strength of its accuracy-to-size ratio (Section 3.4); **cross-encoder/ms-marco-MiniLM-L-12-v2** provides reranking within an acceptable CPU-only latency budget. **spaCy** was chosen for named-entity recognition because the `en_core_web_sm` model is small enough to load in seconds yet recognises the entity types required by the knowledge graph (Section 4.6). **NetworkX** provides the graph data structure, and **Anthropic's Claude** was chosen as the language-model provider for reasons elaborated in Section 2.7.

The choice not to introduce a relational metadata database was deliberate.

The proposal listed PostgreSQL or SQLite as a metadata store; in practice, the JSON-file persistence pattern adopted across the storage layer, document registry, conversation memory, and query history was sufficient and simpler to audit. A database would have added operational complexity without solving any problem the system actually had.

A more substantive technology decision was the choice not to use a high-level RAG framework such as LangChain or LlamaIndex. Both offer turnkey RAG pipelines and would have shortened the path to a working demonstration. They were rejected for two reasons. First, the project brief required demonstrating understanding of the underlying techniques rather than orchestrating opaque framework calls; building hybrid retrieval, RRF fusion, cross-encoder reranking, HyDE, adaptive re-retrieval, grounding scoring, and the answer evaluator from established library primitives produces a system whose behaviour can be inspected, tested, and explained at the level of detail this dissertation requires. Second, framework abstractions impose their own assumptions about pipeline shape, evaluation methodology, and component interfaces, which would have made the architectural decisions described in Chapter 4 either impossible or significantly more constrained. The trade-off is more code to write and maintain in exchange for full control of the system's behaviour; for a dissertation project, the trade-off is the right one.

2.6 Empirical Problem-Solving

Several technical problems encountered during development were resolved empirically rather than from prior knowledge or literature. Two examples illustrate the pattern.

The grounding-scorer initially used term-overlap scoring between answer sentences and retrieved chunks. Manual inspection of test outputs revealed that it inflated to 1.000 on trivial vocabulary matches, producing no useful discrimination between well-grounded and poorly-grounded answers. The solution was to replace the lexical overlap with embedding-cosine similarity, which produces discriminating mid-range values (typically 0.4–0.9 on correctly-grounded answers, dropping below 0.3 on hallucinated content). The same iteration occurred subsequently in the fact verifier; both modules' docstrings carry the rationale explicitly so that the lexical approach is not reintroduced by future maintainers.

A second example involved a runtime error from the `sentence-transformers` library on the development machine: a `NotImplementedError: Cannot copy out of meta tensor` arising from PyTorch's lazy tensor-loading interaction with the library's device-management code. The error did not appear during early development but began occurring after a routine library upgrade. The resolution — explicitly passing `device="cpu"` at model construction to force eager weight-loading

— was found through a combination of stack-trace inspection, library-source reading, and issue-tracker search. The fix is documented inline in `EmbeddingService` and `CrossEncoderReranker` so the reasoning survives any future re-encounter with the same error.

These two cases are representative: the project's progress depended on a steady accumulation of small empirical resolutions, each converted into either a code change with explanatory comments or a test that prevents recurrence.

2.7 Scope Evolution

The system as delivered diverges from the system described in the October 2025 proposal in several substantive respects, and the divergence is large enough to warrant explicit documentation rather than to be absorbed into the surrounding chapters.

The proposal centred on a privacy-first local deployment: a Llama 3 language model running locally without external API access, embeddings fine-tuned on the user's personal writing style, LoRA fine-tuning of the local model for tone and relevance, IMAP email ingestion, and a front-end stack of React with Tailwind and shadcn UI components served by a FastAPI backend with PostgreSQL or SQLite metadata storage. As delivered, the system uses Anthropic's Claude as the language-model provider via the cloud API; embeddings use the off-the-shelf `all-MiniLM-L6-v2` model without fine-tuning; LoRA is not implemented; email ingestion is not implemented; the front end is a twelve-view single-page Flask application using vanilla JavaScript; storage is FAISS plus JSON files rather than Qdrant plus a relational database.

Three of the original ten objectives — the local-only deployment, the LoRA fine-tuning, and the embedding fine-tuning — were deliberately de-scoped during development. Two factors drove this decision. First, building a high-quality evaluation framework (Section 4.7) emerged as the more substantial methodological contribution as the project progressed: the from-scratch RAGAS-style answer evaluator, with its decomposition of faithfulness, answer relevancy, and context precision, provides the rigorous reference-free evaluation that the proposal's success criteria demanded. That evaluator depends on a high-capability LLM functioning both as generator and as judge. Running the full pipeline against a local Llama-class model would have produced lower-quality answers and unreliable judgements, undermining the evaluation chapter's conclusions. Second, LoRA fine-tuning and embedding fine-tuning each require either substantial labelled data or significant GPU compute that was not available within the project's resources, and would have produced narrow improvements at the cost of breadth elsewhere.

The de-scoping decision is therefore best understood as a trade-off between

depth in two different directions. The proposal's depth direction was personalisation through fine-tuning under privacy constraints; the delivered system's depth direction is retrieval sophistication and rigorous evaluation under general-purpose constraints. The features added beyond the proposal — six chunking strategies, hybrid retrieval with RRF, cross-encoder reranking, three query-expansion strategies including HyDE, semantic caching, adaptive re-retrieval, sentence-level grounding, fact verification, knowledge graphs, conflict detection, study-path generation, quiz generation, summarisation, and the two-layer evaluation framework — represent substantially more engineering and research depth than the original proposal's feature set. The revised objectives stated in Chapter 1 reflect what was built, and the evaluation in Chapter 5 measures the system against those revised objectives. The architecture's clean abstractions (Section 4.8) leave the de-scoped features as future work rather than as missing functionality: `LLMProvider` admits a local-Llama implementation; the embedding service can be swapped for a fine-tuned variant; LoRA can be wired in beneath `ClaudeProvider` or its replacement. The scope decision is therefore reversible, which is the strongest defence of the position taken.

Chapter 3

Technology Review

3.1 Introduction and Scope

The system described in this dissertation integrates a substantial number of established techniques from information retrieval, natural language processing, and software engineering. This chapter surveys the technical foundations on which each of those choices rests: lexical and dense retrieval, approximate nearest-neighbour indexing, hybrid retrieval and reranking, document chunking strategies, query expansion, the Retrieval-Augmented Generation (RAG) paradigm itself, and the evaluation methodologies appropriate to such systems. Where the literature is contested or the standard benchmarks are imperfect proxies for the use case at hand, those tensions are surfaced rather than glossed over. Each subsection below is positioned to motivate a concrete design choice that appears in Chapter 4.

3.2 Review Methodology

Sources were located through structured searches of IEEE Xplore, the ACM Digital Library, arXiv, and Google Scholar between February and April 2026. The principal search strings combined retrieval terminology with contemporary qualifiers, for example `("dense retrieval" OR "semantic search") AND (BERT OR transformer)`, `("retrieval-augmented generation" OR RAG) AND (survey OR evaluation)`, and `("approximate nearest neighbour" OR ANN) AND (HNSW OR "product quantisation")`. Backward snowballing from highly cited papers identified foundational works [15, 9, 2], while forward citation tracking captured recent developments in evaluation methodology [8, 6]. Sources were retained where they appeared in peer-reviewed venues (NeurIPS, EMNLP, SIGIR, IEEE TPAMI, ACM TOIS, JOSS) or had attracted significant citation impact in the arXiv preprint literature; sources lacking empirical validation were excluded. Roughly twenty-five

sources met these criteria after deduplication, supplemented by foundational textbook material [15] and the canonical software-engineering references that underpin the methodology chapter [16, 17, 14].

3.3 Lexical Information Retrieval

Lexical or sparse retrieval methods score documents against queries by counting overlapping terms, weighted to discount common words and to favour documents that match rare query terms repeatedly. The Okapi BM25 ranking function remains the canonical instance of this approach and continues to serve as a strong baseline more than two decades after its introduction [9]. BM25 combines term frequency with inverse document frequency under a probabilistic relevance model, with document-length normalisation that prevents long documents from artificially dominating ranked output. The function is fast to compute, requires no training data, and produces interpretable scores; its term inverted index is straightforward to maintain incrementally as documents are added and removed.

The principal weakness of lexical retrieval is the *vocabulary mismatch* problem [15]: when a user's query and a relevant document use different words to describe the same concept, BM25 returns no overlap and the document goes unretrieved. A query for "automobile pricing" will not match a document discussing "car costs" under any setting of the BM25 parameters, however well-tuned. This is not a tractable problem within the lexical paradigm, since exhaustive synonym enumeration cannot anticipate every paraphrase. The limitation provides the central motivation for dense semantic retrieval.

3.4 Dense Sentence Embeddings

Dense retrieval addresses vocabulary mismatch by mapping text to vectors in a high-dimensional space where geometric proximity corresponds to semantic similarity, rather than to lexical overlap. The Transformer architecture provided the foundation [18], and BERT demonstrated that bidirectionally pre-trained Transformers produce state-of-the-art representations across a wide range of downstream tasks [19]. However, BERT's native cross-encoder architecture is computationally infeasible for large-scale retrieval: a single query against ten thousand candidate passages would require ten thousand independent BERT forward passes.

Sentence-BERT (SBERT) resolves this by fine-tuning BERT in a siamese configuration on natural language inference pairs, producing fixed-size sentence embeddings that can be compared in constant time using cosine similarity [10]. Reimers and Gurevych report that the time required to identify the most similar

pair within a collection of ten thousand sentences fell from approximately sixty-five hours under naive BERT to roughly five seconds under SBERT, while pairwise semantic textual similarity correlation improved from 0.548 to 0.749 Spearman. The Personal Knowledge Assistant uses the `all-MiniLM-L6-v2` SBERT variant, which trades some accuracy for a 384-dimensional embedding (versus 768) and a model footprint under ninety megabytes — suitable for the lightweight, locally-runnable deployment constraints of the project.

Dense Passage Retrieval (DPR) extended this idea to open-domain question answering by training separate query and passage encoders contrastively [20]. DPR raised top-twenty retrieval accuracy on the Natural Questions benchmark from 59.1% under BM25 to 78.4% — a 19.3 percentage-point improvement that established dense retrieval as the dominant paradigm for question answering.

Critical Assessment The dense retrieval literature disproportionately reports results on Wikipedia-derived benchmarks such as Natural Questions [20] and TriviaQA. Performance gains measured on factoid questions over encyclopaedic prose may not transfer to domains where exact terminology carries semantic weight: legal retrieval distinguishes "shall" from "may", medical retrieval treats brand-name synonyms as clinically distinct, and source-code search is exquisitely sensitive to identifier exactness. The system described here therefore treats dense retrieval as one signal among several rather than as a complete replacement for lexical methods — a position elaborated in Section 3.6.

3.5 Approximate Nearest-Neighbour Search

Once documents are represented as dense vectors, retrieval becomes a nearest-neighbour problem: given a query vector, return the k stored vectors with the highest cosine similarity. Exact search by exhaustive comparison is acceptable for small collections but scales linearly in the number of vectors and in the dimensionality, becoming prohibitive at the billion-scale collections that characterise production deployments. Approximate nearest-neighbour (ANN) algorithms trade exactness for orders of magnitude in query-time speedup.

Hierarchical Navigable Small World graphs (HNSW) construct a multi-layer proximity graph in which upper layers contain exponentially fewer vertices connected by long-range edges, while lower layers provide dense local coverage [1]. A search descends from a fixed entry point, navigating greedily by similarity at each layer; the construction achieves logarithmic average-case complexity with reported recall exceeding 95% at sub-millisecond latency on the SIFT1M benchmark. Product quantisation (PQ) takes a complementary approach, decomposing each vector into m subvectors and quantising each subvector separately [2]. A 128-dimensional

Table 3.1: Comparative trade-offs of approximate nearest-neighbour approaches. Ranges reflect cross-study variation in hyperparameters and hardware [1, 2, 3].

Method	**Recall@10**	**Memory**	**Query Time**
HNSW	95–99%	High	< 1 ms
PQ (64 bytes)	70–85%	Very low	10–50 ms
IVFPQ	85–95%	Low	1–10 ms
Flat (exact)	100%	Very high	$O(N)$

float32 vector occupying 512 bytes can be compressed to 8 bytes under an 8-codebook PQ — sixty-four times reduction in memory at the cost of recall@10 in the 70-85% range. Inverted file structures combined with PQ (IVFPQ) sit between these extremes [3], while the Faiss library provides production implementations of all three families [21]. Table 3.1 summarises the trade-off space.

For collections of the order described here — low thousands of chunks per individual user — the asymptotic advantages of approximate methods become negligible, and the system uses Faiss's `IndexFlatIP` for exact inner-product search over $L2$-normalised vectors. This delivers true cosine similarity with no recall loss whatsoever, while consuming a few megabytes of memory at the corpus sizes characteristic of personal knowledge bases.

3.6 Hybrid Retrieval and Reciprocal Rank Fusion

The complementary failure modes of lexical and dense retrieval invite hybrid approaches that exploit both. BM25 is strong on exact matches of rare terms, technical identifiers, and proper nouns; dense retrieval captures paraphrase and conceptual similarity but can be misled by surface-level similarities that do not reflect semantic relevance to the query. Combining the two consistently outperforms either alone, but the combination is non-trivial because the underlying scores live on incomparable scales: BM25 produces unbounded positive numbers, dense cosine similarity is bounded in $[-1, 1]$, and direct linear interpolation requires either dataset-specific tuning or score normalisation that introduces its own distortions.

Reciprocal Rank Fusion (RRF) avoids the score-normalisation problem entirely by operating on *ranks* rather than scores [11]. For each retrieved document d, RRF computes a fused score

$$\mathrm{RRF}(d) = \sum_{i \in S} \frac{1}{k + r_i(d)} \tag{3.1}$$

where S is the set of retrievers, $r_i(d)$ is the rank of d in the output of retriever i, and k is a damping constant typically set to 60. Documents retrieved highly by multiple

systems accumulate large fused scores, while a document that appears only in one ranked list still contributes a modest amount. Cormack et al. demonstrated that RRF outperformed both individually trained Condorcet-style rank aggregation and individual rank learning methods on standard TREC tasks, despite having no learned parameters whatsoever. The simplicity of the formula and its independence from raw score distributions are the principal reasons RRF has become a de facto standard for production hybrid retrieval systems.

3.7 Cross-Encoder Reranking

Bi-encoder retrieval is fast because document and query embeddings are computed independently, but the resulting score is only a coarse proxy for genuine query–passage relevance. Cross-encoders, in contrast, take a concatenated query–passage pair as a single input and produce a relevance score conditioned jointly on both [12]. The joint conditioning enables fine-grained interaction between query terms and passage tokens through the transformer's attention mechanism, yielding more accurate relevance judgements — at the cost of being infeasible to apply to every document in the collection.

The natural compromise is a two-stage architecture: a fast bi-encoder retrieves a candidate pool of perhaps twenty to one hundred passages, and a slower cross-encoder reranks that pool to produce the final ordering. Nogueira and Cho reported substantial precision gains on the MS MARCO passage ranking task using a BERT-based cross-encoder, an approach that has since been widely adopted in production retrieval pipelines. The Personal Knowledge Assistant uses `cross-encoder/ms-marco-MiniLM-L-12-v2`, a distilled cross-encoder model approximately one hundred and thirty megabytes in size that reranks pools of around twenty candidates within a few hundred milliseconds on commodity CPU hardware.

3.8 Document Chunking Strategies

Documents must be split into smaller passages before embedding because sentence-transformer models impose a strict token limit, typically 256 or 512 tokens, beyond which input is truncated; equally importantly, embedding an entire document into a single vector loses the granularity required for precise retrieval. The choice of chunking strategy is therefore not a peripheral implementation detail but a primary determinant of retrieval quality, and the literature describes a spectrum of approaches with substantively different trade-offs.

Fixed-size chunking divides text into uniform blocks at a fixed character or

token count, optionally with a small overlap between consecutive chunks. The approach is fast and predictable but cuts indiscriminately through sentence and paragraph boundaries, producing chunks that begin and end mid-clause. *Sentence-boundary chunking* respects punctuation, packing complete sentences into chunks until the size limit is reached. Both methods are lexical: they make no use of the content's semantic structure.

Semantic chunking strategies attempt to align chunk boundaries with genuine topic shifts. *Embedding-similarity chunking* embeds each sentence and inserts a chunk boundary wherever the cosine similarity between consecutive sentences drops below a threshold, on the principle that an abrupt similarity drop signals a topic change. *Density clustering* via HDBSCAN groups sentence embeddings into variable-density clusters [22], optionally after dimensionality reduction with UMAP [23], with chunk boundaries placed at cluster transitions. *Topic modelling*-based chunking, exemplified by BERTopic [24], uses a class-based TF-IDF extension over clustered embeddings to assign each sentence a discrete topic label, again segmenting at topic boundaries.

Methodological Tensions Empirical comparisons of chunking strategies do exist — Yepes et al. benchmark element-aware against paragraph-level chunking on the FinanceBench corpus and report substantial retrieval-quality gains from structure-aware segmentation [25] — but the field lacks a canonical cross-domain benchmark that controls for embedding model, retriever, and downstream evaluation simultaneously. Cross-paper comparisons are therefore unreliable, and reported gains are typically corpus-specific. The Personal Knowledge Assistant therefore implements all six strategies discussed above and exposes them through a uniform configuration interface, allowing empirical comparison on the same corpus rather than relying on cross-paper claims of superiority.

3.9 Query Expansion and Hypothetical Document Embeddings

Query expansion enriches a user's original query with additional terms or reformulations to improve retrieval recall, particularly when the query is short or uses different vocabulary from the relevant documents. Classical techniques include synonym substitution from lexical resources such as WordNet, and pseudo-relevance feedback, in which terms from initially retrieved documents are appended to the query for a second retrieval pass. Both have a long history in information retrieval but are limited by the static nature of the lexical resources they depend on.

The advent of capable generative language models enabled a more flexible strat-

egy: Hypothetical Document Embeddings (HyDE) [26]. Rather than expanding the query directly, HyDE asks an instruction-tuned language model to generate a short hypothetical document that would plausibly answer the query, embeds that hypothetical document with the same dense encoder used for the corpus, and uses the resulting embedding as the retrieval query. The intuition is that a hypothetical answer lives in approximately the same region of embedding space as real documents on the topic — closer than the query itself often is, particularly for short or ambiguous queries. Gao et al. reported substantial zero-shot retrieval gains across multiple benchmarks, with the dense encoder acting as a noise filter on factual hallucinations introduced by the generator.

3.10 Retrieval-Augmented Generation

Retrieval-Augmented Generation [5] combines a parametric generator with a non-parametric memory accessed via dense retrieval, building on the joint pre-training paradigm introduced by REALM [27]. The original RAG formulation paired a BART seq2seq model with a DPR-style retriever over a Wikipedia index; for each input, the retriever selects the top-k relevant passages, and the generator conditions its output on both the input and the concatenated retrieved context. RAG addresses three persistent weaknesses of pure parametric language models: the inability to update knowledge without retraining, the tendency to hallucinate plausible-sounding falsehoods [4], and the absence of explicit source attribution. Lewis et al. reported state-of-the-art performance on open-domain question answering benchmarks while providing the citations that pure parametric models cannot.

The RAG paradigm has since differentiated. Gao et al. characterise the evolution as moving from *Naive RAG* — the basic retrieve-then-generate pipeline — to *Advanced RAG*, which adds pre-retrieval and post-retrieval optimisations such as query rewriting, query expansion, re-ranking, and context compression, and finally to *Modular RAG*, which decomposes the system into interchangeable modules with flexible control flow [6]. The system described in this dissertation is firmly in the Advanced category, integrating semantic caching, conversational query rewriting, HyDE, hybrid retrieval with reranking, and confidence-driven adaptive re-retrieval into a coherent pipeline.

Conflicting Findings The RAG literature is not unanimous in its assessments. Gao et al. emphasise consistent gains in knowledge-intensive tasks and present RAG as a stable, deployable architecture [6]. Empirical robustness studies paint a more cautious picture: Chen et al. introduce the Retrieval-Augmented Generation Benchmark (RGB) and report that contemporary language models exhibit

measurable weaknesses across noise robustness, negative rejection (declining to answer when no relevant context is retrieved), information integration across multiple passages, and counterfactual robustness against deliberately incorrect context [7]. The disagreement is partly definitional — "RAG" covers everything from the basic Lewis architecture to elaborate pipelines with multiple retrieval rounds — and partly methodological, since evaluation protocols vary across studies. The practical implication is that RAG benefits should be reported with sufficient context to support comparison, and evaluated on a corpus and task representative of intended deployment rather than on benchmark defaults alone.

3.11 Evaluation of Retrieval-Augmented Systems

RAG evaluation presents a fundamental difficulty: a system that retrieves imperfect context but generates a correct answer is arguably superior to one that retrieves perfect context and generates a poor answer. Evaluation must therefore address two distinct dimensions — the quality of the retrieval stage and the quality of the generated answer — and the literature has converged on largely separate methodologies for each.

Retrieval evaluation continues to use the classical information-retrieval metrics. Precision-at-K measures the fraction of the top K retrieved items that are relevant; Mean Reciprocal Rank (MRR) measures the rank of the first relevant item; Normalised Discounted Cumulative Gain (nDCG@K) accounts for the position of relevant items within the ranked list using a logarithmic discount [13]. All three require a labelled judgement set that maps queries to known-relevant document or chunk identifiers; this labelled set is laborious to produce but, once established, supports reproducible quantitative comparison of retrieval changes.

Answer evaluation is more contested. Reference-based methods such as BLEU and ROUGE compare generated output to a single ground-truth answer, but penalise valid paraphrases and miss factual errors that preserve surface form. The RAGAS framework [8] addresses this by proposing three reference-free metrics computed using a language model as judge: *faithfulness*, the fraction of atomic claims in the answer supported by the retrieved context, captures hallucination; *answer relevancy*, computed by generating reverse-questions from the answer and measuring their similarity to the original question, captures whether the answer is on-topic; and *context precision*, a rank-weighted measure of how usefully the retrieved chunks were ordered relative to the ground-truth answer, captures retrieval quality at the generation-relevant level.

Critical Assessment LLM-as-judge evaluation introduces a methodological tension that the RAGAS framework itself does not fully resolve. When the same

model family judges both the answer it generated and the context it was given, same-family bias can inflate faithfulness scores: the judge may be primed to accept outputs in a style and vocabulary similar to its own. Zheng et al. document the broader risks of LLM-as-judge protocols, including verbosity bias, position bias, and self-preference, and recommend that LLM-as-judge results be calibrated against human ratings on a representative sample [28]. The system described here uses Claude as both generator and judge — a deliberate choice for cost and consistency reasons, but one whose limitations are addressed honestly in Chapter 5.

3.12 Knowledge Graphs from Unstructured Text

Knowledge graphs represent information as nodes (entities) connected by edges (relationships), supporting structured queries and visual exploration that purely vector-based representations do not. Constructing a knowledge graph from a corpus of unstructured documents typically proceeds in two stages: named-entity recognition (NER) extracts mentions of people, organisations, locations, and other typed entities, and a relation extraction step links entities by their co-occurrence or by an explicitly classified semantic relation.

The spaCy library provides production-quality pre-trained NER models with typed entity labels — `PERSON`, `ORG`, `GPE`, `PRODUCT`, and others — under the same Python interface that handles tokenisation, dependency parsing, and lemmatisation [29]. NetworkX [30] is the canonical Python library for graph construction and analysis, supporting weighted edges, connected-component analysis, centrality measures, and serialisation to standard interchange formats. The Personal Knowledge Assistant constructs a co-occurrence graph in which entities are nodes, edges connect entities that appear in the same chunk, and edge weight reflects the number of co-occurrences across the corpus. This produces a visually navigable map of the corpus's conceptual structure that complements semantic search, particularly for exploration of unfamiliar material.

3.13 Synthesis

The literature reviewed in this chapter establishes the design space the Personal Knowledge Assistant occupies: dense retrieval over sentence embeddings is the default, but lexical retrieval remains valuable for rare terms; cross-encoder reranking refines bi-encoder candidates; HyDE mitigates the query–document representation gap; the Advanced RAG paradigm structures the generation pipeline; classical IR metrics measure retrieval quality and reference-free LLM-as-judge methods measure answer quality. These choices are not independent — hybrid retrieval implies

both indices, reranking presupposes a candidate pool, RAGAS evaluation presupposes a generator and a ground-truth set — and the system design described in Chapter 4 composes them into a single coherent pipeline with the abstractions necessary to swap any one component without disturbing the others.

A useful concrete reference point is LangChain, the most widely used open-source RAG framework. LangChain's default `RetrievalQA` chain provides dense retrieval over a configurable embedding model followed by an LLM generation call — the basic Naive RAG pattern. Hybrid retrieval, cross-encoder reranking, query expansion, grounding analysis, and adaptive re-retrieval are available as add-on modules but must be wired in by the developer; the framework provides components, not an opinionated end-to-end pipeline. The Personal Knowledge Assistant goes further than this default: every component LangChain offers as optional is integrated into a single coherent pipeline with explicit abstraction boundaries that allow individual components to be swapped or benchmarked. The trade-off is direct — more code to write and maintain in exchange for full control of the system's behaviour and evaluability.

Chapter 4

System Design

4.1 Architectural Overview

The Personal Knowledge Assistant is structured as a six-layer architecture in which each layer depends only on the layer immediately beneath it, exposing a stable interface upward. This discipline keeps changes local: replacing the embedding model affects the ingestion and retrieval layers but not the pipeline above them; swapping the language-model provider affects only the abstraction boundary described in Section 4.5. The layered structure is shown in Figure 4.1.

The ingestion layer reads raw documents from disk and produces `Chunk` objects with embeddings and named-entity annotations. The storage layer persists chunks and embeddings into FAISS and BM25 indices alongside a document registry that tracks file hashes for change detection. The retrieval layer combines the two indices via Reciprocal Rank Fusion and applies cross-encoder reranking. The RAG pipeline layer orchestrates the eleven-step query lifecycle described in Section 4.5, including caching, conversation memory, and post-generation grounding analysis. The intelligence layer composes the lower layers into higher-order features — knowledge graphs, conflict detection, study path generation. The interface layer presents three parallel front ends sharing the same backend: a command-line interface, a Flask web application implementing a twelve-view single-page UI, and a Python API for programmatic use. The remainder of this chapter describes each layer in turn, with concrete class and method references that map directly to the source repository (see Appendix A).

4.2 Ingestion Pipeline

The ingestion pipeline transforms a heterogeneous mix of input documents into a uniform internal representation. `DocumentManager` implements a Factory pat-

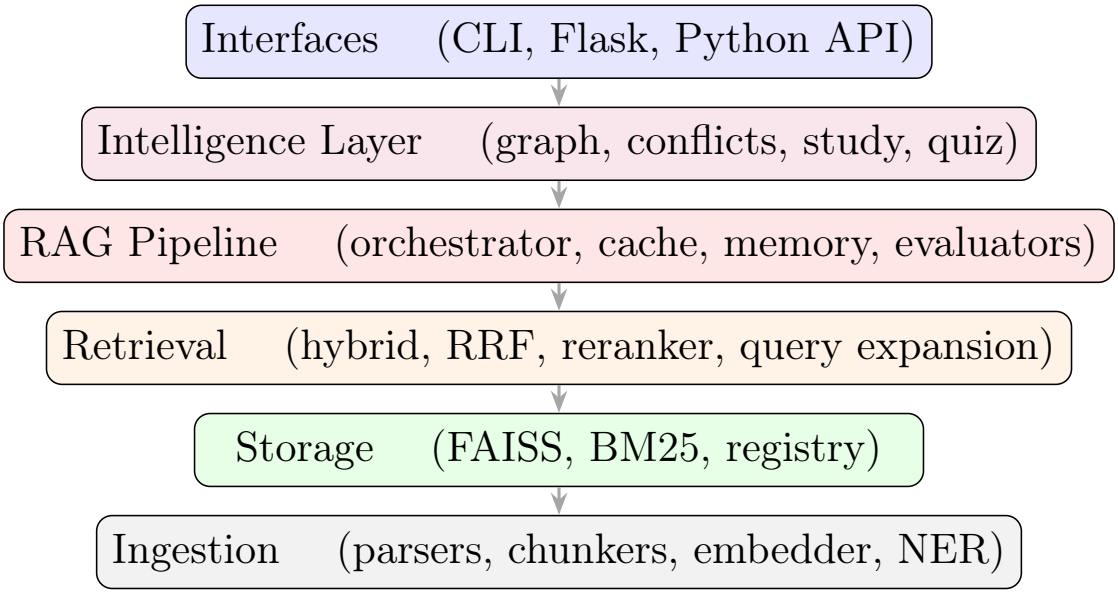

Figure 4.1: Six-layer architecture. Each layer depends on the layer below. Cross-cutting concerns (logging, configuration) span the stack.

tern [16] over three concrete parsers: `PDFParser`, `DOCXParser`, and `TextParser` (which handles Markdown and plain-text uniformly). Each parser inherits from `BaseParser` and is required to implement only `parse()`; shared functionality including metadata extraction and deterministic document-ID generation lives on the abstract base, an instance of the Template Method pattern. New file-format support amounts to writing one new parser class — typically between sixty and two hundred lines depending on the format's structural complexity — and registering it in `DocumentManager.parsers`.

Once a document is parsed, it is handed to a chunker selected via `ChunkManager`, a second Factory whose responsibility is mapping a configuration string (e.g. `"sentence"`, `"density_clustering"`) to a concrete chunker instance. Six strategies are implemented, summarised in Table 4.1. The distinction between simple lexical strategies (Fixed, Sentence) and semantic strategies (the four ML-backed approaches) is reflected in their initialisation costs — semantic chunkers lazily load the `sentence-transformers` model on first use, deferring a model load of approximately ninety megabytes until it is actually needed.

After chunking, `NERExtractor` annotates every chunk with named entities using spaCy's `en_core_web_sm` model [29], filtered to nine relevant labels (`PERSON`, `ORG`, `GPE`, `LOC`, `PRODUCT`, `EVENT`, `WORK_OF_ART`, `LAW`, `NORP`). These annotations serve a dual purpose: they support entity-aware result boosting at retrieval time

Table 4.1: The six implemented chunking strategies. `ChunkManager.recommend_strategy()` chooses among them based on document length and section count.

Strategy	Mechanism	ML dependency	Best for
`fixed`	Character-window with overlap	none	Baseline; very fragmented text
`sentence`	Pack sentences to size limit	none	Well-written prose; short docs
`embedding_similarity`	Cut where consecutive-sentence cosine drops	MiniLM embedder	Topic-shifting medium docs
`density_clustering`	HDBSCAN over (UMAP-reduced) embeddings	MiniLM + HDBSCAN [22]	Long docs with clear topics
`topic_modeling`	BERTopic [24] segmentation	MiniLM + BERTopic	Multi-topic documents needing labels
`recursive_hierarchical`	Document → section → paragraph tree	MiniLM (optional)	Structured documents with sections

(Section 4.4) and provide the node set for the knowledge graph (Section 4.6). `EmbeddingService` then computes 384-dimensional embeddings via `all-MiniLM-L6-v2`, with the model lazily loaded under `device="cpu"` to circumvent a known PyTorch meta-tensor incompatibility on certain version combinations. The same `EmbeddingService` instance is shared across the `KnowledgeBase`, `GroundingScorer`, and `FactVerifier` via a property exposed on `KnowledgeBase` — preventing the model from being loaded three times into memory.

Finally, `DocumentRegistry` hashes the raw file bytes with SHA-256 and stores the hash alongside the document ID. On subsequent ingest calls, an unchanged hash short-circuits the parse–chunk–embed pipeline: the document is recognised as already current, and re-ingestion is silently skipped. This makes the system idempotent under repeated ingestion of the same corpus directory — a convenience for users who periodically refresh their knowledge base.

4.3 Storage Layer

The storage layer maintains two indices over the same chunks. The vector store `FAISSVectorStore` wraps a Faiss `IndexFlatIP` [21]. Inner-product search over L^2-normalised vectors is mathematically equivalent to cosine similarity but avoids the runtime division; embeddings are normalised once on insertion using `faiss.normalize_L2` and queries are normalised at search time. For the corpus sizes characteristic of single-user knowledge bases — typically thousands of chunks — exact search runs in well under one millisecond and incurs none of the recall loss associated with approximate methods such as HNSW or IVFPQ (Section 3.5). The trade-off becomes unfavourable only for collections substantially larger than the personal-use scope.

The lexical store `BM25Store` wraps the `rank-bm25` library, exposing the same `search(query, top_k, filter_doc_id)` interface as the vector store. Both stores share the `VectorStore` abstract interface and the `SearchResult` value object, so any retrieval code can treat them interchangeably — an instance of the Strategy pattern.

The registry layer `DocumentRegistry` closes the loop on persistence. Indices, registry, and metadata are saved as a coordinated file set with a shared base path: `<path>.faiss`, `<path>.meta.json`, `<path>.bm25.json`, and `<path>.registry.json`. Loading reverses this process atomically. Conversation memory, semantic cache, query history, and annotations follow the same disciplined save/load convention with their own JSON files, so the entire system state can be persisted, copied, or audited through ordinary file operations. This treatment of persistence as an infrastructure detail rather than as a coupling between business logic and storage is consistent with the architectural-boundary discipline described by Martin [17]: the domain layer constructs `Chunk` and `SearchResult` objects without knowing whether they will ultimately be stored, and the storage layer accepts those objects without knowing how they were produced.

4.4 Retrieval Layer

The retrieval layer composes the two storage backends into a unified search interface. `HybridSearcher` runs the BM25 and FAISS searches independently, each returning a candidate pool of twenty results, then merges them using Reciprocal Rank Fusion [11] with the canonical damping constant $k = 60$. The merge logic in `reciprocal_rank_fusion()` accumulates each chunk's score across both ranked lists according to Equation (3.1) and re-ranks by the fused total. Three early-return branches handle degenerate cases: both stores empty, only FAISS empty, and only BM25 empty — avoiding division-by-zero or empty-merge errors.

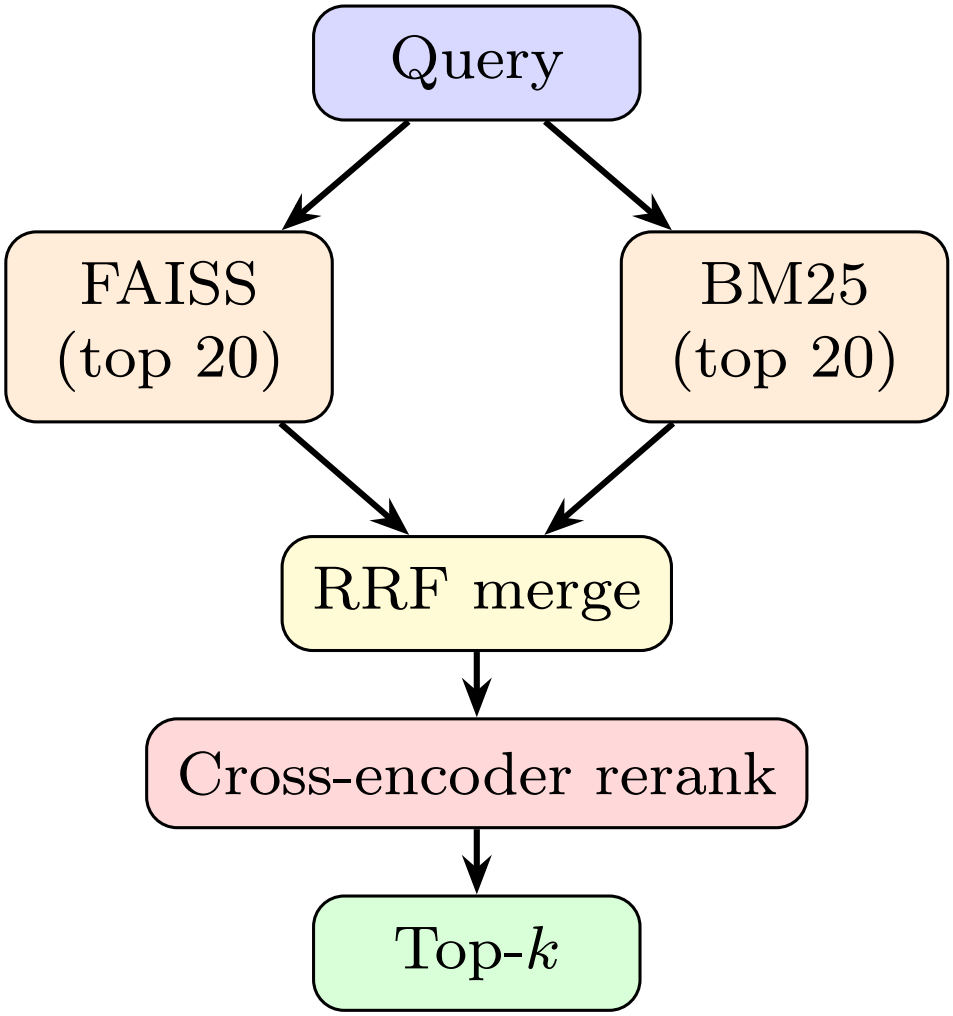

Figure 4.2: Hybrid retrieval. Two parallel search paths fuse via RRF; the fused list is re-scored by a cross-encoder.

After fusion, `CrossEncoderReranker` re-scores the merged pool using `cross-encoder/ms-marco-MiniLM-L-12-v2`, producing joint-conditioning relevance scores that refine the bi-encoder approximations from FAISS [12]. The cross-encoder is loaded lazily and pinned to CPU to avoid the same meta-tensor issue described above. Two complementary refinements operate alongside the core hybrid pipeline. `QueryExpander` implements three expansion strategies — WordNet synonym substitution, heuristic multi-query rephrasing, and template-based HyDE [26] for environments without an available generator — selectable via configuration. `boost_by_entities()` provides entity-aware reranking: chunks whose pre-extracted NER labels overlap the query's named entities receive a small additive bonus per overlap, with an op-

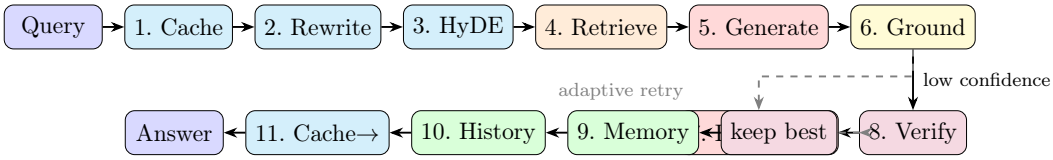

Figure 4.3: Eleven-stage RAG pipeline. Solid arrows: main path. Dashed arrows: the adaptive-retry loop, which re-runs retrieval with relaxed parameters when the grounding score is below threshold; the highest-confidence attempt across retries is kept.

tional hard filter on a specific entity label. The retrieval sequence is summarised in Figure 4.2.

4.5 The RAG Pipeline

`RAGPipeline` is the central orchestrator of the system. Its `query()` method coordinates eleven distinct stages, each delegating to a specialised collaborator object. The pipeline implements the Mediator pattern [16]: collaborators do not communicate directly with one another, only through the pipeline. This isolation is what makes individual components replaceable in testing and what keeps the data flow legible.

Figure 4.3 shows the full flow. Stage 1 consults `SemanticCache`, which embeds the query and looks for any previously cached query whose embedding has cosine similarity above a threshold; a hit returns the cached answer immediately, bypassing the remainder of the pipeline. Stages 2 and 3 transform the query: if recent conversation history contains the topic referent, the query is rewritten into a standalone form (so a follow-up of "what about its risks?" is expanded using the previous question's subject); for the dense retrieval path, `QueryExpander` optionally produces a HyDE expansion. Stage 4 invokes `HybridSearcher` as described in Section 4.4, with the cross-encoder rerank applied inside it. Stage 5 calls the configured `LLMProvider` — currently `ClaudeProvider`, which uses the Anthropic Python SDK under the abstraction — with the retrieved chunks formatted into a context block and the system prompt set to require source-cited answers.

Stages 6 and 7 are the system's principal defence against retrieval failure. `GroundingScorer` computes a sentence-level confidence score: each sentence in the generated answer is embedded, each chunk in the retrieved context is embedded (using the shared `EmbeddingService` instance), and the per-sentence support score is the maximum cosine similarity over all chunks. The overall grounding confidence is the mean of per-sentence supports. If this falls below a configurable threshold, the pipeline re-retrieves with relaxed parameters — a higher candidate

pool, a different query expansion strategy, or a HyDE pass if not previously enabled — and regenerates. A subtle but important detail: across retry attempts the pipeline keeps the *highest-confidence* result rather than the latest, so a noisy retry that produces a worse answer cannot displace a better earlier attempt. The maximum number of retries is bounded; if the bound is reached without crossing the threshold, the best attempt so far is returned with the lower confidence flagged in the response payload.

Stage 8 runs `FactVerifier`, which decomposes the answer into atomic factual claims (with the LLM acting as decomposer) and verifies each claim against the retrieved context using embedding similarity — the architectural complement to the LLM-judge faithfulness metric used in evaluation, but operating live during query handling rather than post-hoc. Stages 9 and 10 persist conversation state: `ConversationMemory` appends the question/answer pair to a sliding window of recent turns (windowed for the LLM context but full history retained on disk), and `QueryHistory` logs every query with timing and confidence metadata to the analytics view in the web UI. Stage 11 stores the final answer back into the semantic cache so future near-duplicate queries hit the fast path.

The grounding scorer's design reflects an iteration the system actually went through. The first implementation used term-overlap scoring, which inflated to 1.000 on trivial vocabulary matches and produced no useful discrimination. The current embedding-cosine implementation produces discriminating mid-range values — typically 0.4 to 0.9 on correctly-grounded answers, dropping below 0.3 when the LLM hallucinates content not in the context. The same iteration occurred in `FactVerifier`; both modules' docstrings carry the rationale explicitly so that future maintainers understand why the lexical approach is not present in the code.

4.6 Intelligence Layer

The intelligence layer composes the lower layers into higher-order features. `GraphBuilder` consumes the same chunks already in the knowledge base and builds a co-occurrence graph using NetworkX [30]: each named entity becomes a node weighted by chunk-frequency, and an edge is created between any two entities that appear in the same chunk, with edge weight reflecting co-occurrence count. The resulting graph is rendered as an interactive D3.js force-directed visualisation in the web UI. The choice of co-occurrence over learned relation extraction reflects a deliberate trade-off: co-occurrence is shallow (it cannot distinguish "X criticised Y" from "X agreed with Y") but produces results immediately on any corpus without the supervision data that relation extractors require. The limitation is acknowledged honestly in Section 4.10.

`ConflictDetector` addresses a failure mode that simple RAG systems silently

exhibit: when two source documents disagree, the pipeline chooses the highest-scoring chunk and never surfaces the disagreement. The detector retrieves the top-N chunks for a topic, groups them by source document, and submits each pairwise document combination to the LLM with a structured-output prompt requesting a JSON conflict verdict. The Pipeline pattern — retrieve, group, compare, report — gives the implementation linear extensibility: each stage's output is the next stage's input, with no hidden dependencies.

`Summariser`, `QuizGenerator`, and `PathGenerator` (study path) follow the same architectural template: retrieve grounded content from `KnowledgeBase`, synthesise via an LLM call with a strict JSON system prompt, validate the response, and return a structured dataclass. Each is approximately three hundred lines and adds no new infrastructure — they are pure compositions of the lower layers, which is what makes the architecture defensible: the heavy work happens once, in retrieval and the pipeline, and the intelligence features ride on top.

4.7 Two Independent Evaluation Layers

Evaluation is implemented as two independent modules with different purposes and dependencies. `src.evaluation.metrics` provides the classical IR metrics — `precision_at_k`, `mean_reciprocal_rank`, `ndcg_at_k` [13] — as pure functions operating on lists of document IDs. They have no LLM dependency and run in microseconds, which makes them ideal for regression testing during development: a unit test can compute Precision@5 on a fixed test query set and fail the build if it regresses. `EvaluationSuite` orchestrates these metrics over a labelled JSON test file mapping queries to known-relevant document IDs, and exposes the results as an `EvaluationReport` with per-query and aggregate scores.

`src.evaluation.answer_eval` implements the second layer: `AnswerEvaluator`, a from-scratch RAGAS-style answer evaluator [8] using the same `LLMProvider` abstraction the pipeline itself uses. Three reference-free metrics are computed per question: *Faithfulness*, the fraction of decomposed atomic claims that are entailed by the retrieved context; *Answer Relevancy*, the mean cosine similarity between the original question and reverse-questions the LLM generates from the answer alone; and *Context Precision*, the rank-weighted precision of retrieved chunks against a ground-truth answer. The composite metric is the harmonic mean of the three, chosen deliberately because the harmonic mean penalises imbalance — a single weak metric drags the composite down, which is the desired behaviour for a quality indicator that should not be gamed by sacrificing one dimension.

The two-layer design separates concerns that would otherwise conflict. The classical IR layer is fast, deterministic, and free; it answers the question *has retrieval quality regressed?* The RAGAS layer is slow and requires a paid LLM

API; it answers the question *is the system producing faithful, on-topic answers?* These are independent failure modes, and conflating them would either prevent fast regression testing (if every test required an LLM call) or would leave generation quality unmeasured (if only the IR layer existed). The cost asymmetry is significant: Precision@5 on twelve queries completes in milliseconds, while a full RAGAS evaluation across the same set requires roughly forty-eight LLM calls and several minutes of wall time.

Automated Verification The architecture is verified by an automated test suite of over five hundred unit and integration tests across twenty-nine test modules, covering parsers, every chunker, the embedding service, both storage backends, hybrid retrieval, the RAG pipeline (including the empty-knowledge-base edge case), the grounding scorer, the fact verifier, the answer evaluator, the knowledge graph, the conflict detector, and the web API endpoints. Tests use synthetic corpora and mocked LLM providers, allowing the full suite to run in seconds rather than the minutes a real LLM evaluation would require. This rapid feedback was essential during development: each architectural change was guarded by tests at every affected layer, and refactoring became safe by construction rather than by inspection.

4.8 Design Pattern Catalogue

The system applies eleven classical design patterns, summarised in Table 4.2. The table lists each pattern with its concrete embodiment in the codebase; readers can verify each entry by inspecting the named class. Pattern usage is not a goal in itself, but the catalogue makes the architectural reasoning legible: every pattern is load-bearing rather than decorative.

The Strategy pattern is the most consequential. Three independent configurability points — LLM provider, chunking strategy, parser type — share the same shape: a small interface, multiple concrete implementations, factory-driven selection. The benefit becomes concrete when a component changes: replacing the language-model backend at one point during development required modifying only `LLMProvider` and the configuration entry, leaving every call site that depends on its output unchanged. Without the abstraction, the same change would have rippled through every module that constructs prompts or consumes generated content. The cost of designing the interface in advance is repaid the first time the underlying technology is replaced.

Table 4.2: Design patterns used in the system, with concrete classes from the source repository.

Pattern	Concrete embodiment	Role
Strategy	`LLMProvider`, `BaseChunker`, `BaseParser`	Interchangeable algorithm families
Factory	`DocumentManager`, `ChunkManager`	Routing input to the correct concrete strategy
Mediator	`RAGPipeline`	Orchestrates collaborators that do not talk directly
Repository	`DocumentRegistry`, `QueryHistory`, `AnnotationStore`	Persistent collection abstraction
Proxy	`SemanticCache`	Transparent intercept of expensive operations
Builder	`GraphBuilder`	Stepwise construction of a complex object
Facade	`KnowledgeBase`, `PathGenerator`	Simplified entry point over many subsystems
Template Method	`Summariser`, `BaseParser`	Fixed skeleton with overridable steps
Pipeline	`ConflictDetector`	Linear stages, each consuming the previous output
App Factory	`create_app()` in Flask	Configurable instance construction
Lazy Loading	cross-encoder, spaCy, sentence-transformers, Anthropic client	Defer expensive resource acquisition

4.9 Interface Layer

The interface layer presents three parallel front ends sharing a single backend. The CLI (`src.cli`) exposes eight commands for ingestion, search, query, evaluation, and graph operations — intended for scripted use and reproducible evaluation runs. The Python API is the codebase itself: any class is importable, and a third-party application can construct `KnowledgeBase` and `RAGPipeline` directly. The Flask web application (`src.web`) provides the primary user-facing interface; its factory function `create_app()` registers six blueprints that partition the API surface by responsibility: `core` (health, frontend routes, document management), `chat` (RAG-powered Q&A and streaming), `search` (semantic search and document upload), `study` (study paths, quizzes, summaries), `intelligence` (knowledge graph, conflicts, similarity, evaluation), and `analytics` (query history and

annotations). The blueprint partition follows the Single Responsibility Principle [17]: each blueprint can be opened, understood, and modified in isolation.

4.10 Design Tensions

Three architectural decisions warrant explicit acknowledgement, both because they are imperfect and because the brief specifically rewards identifying limitations as evidence of insight.

First, six chunking strategies is more than the system strictly requires. A production deployment would choose one or two and remove the others. The wide implementation supports the empirical chunking comparison referred to in Section 3.8 and gives the system credibility as a research artefact, but a leaner architecture is defensible. The cost of the broad implementation is concentrated in maintenance rather than runtime — the lazy-loading discipline ensures unused chunkers consume no memory — so the trade-off is acceptable within the scope of this dissertation.

Second, the knowledge graph uses entity co-occurrence rather than learned relation extraction. Co-occurrence is structurally weaker: "X criticised Y" and "X agreed with Y" produce identical edges. A relation-extraction module fine-tuned on a corpus-specific schema would produce richer typed edges but would require labelled training data the system does not have. The current implementation is a defensible default for unsupervised settings; replacing it with a learned extractor is a clean extension point because `GraphBuilder` is the sole consumer of the chunk content for graph purposes.

Third, the answer evaluator uses Claude as both generator and judge. This was a deliberate cost-and-consistency decision but introduces the same-family bias documented in Section 3.11. The architecture mitigates this by routing both calls through `LLMProvider`, so a future evaluation pass with a different judge model (e.g. a local Llama-class model or GPT) requires only a configuration change, not a code change. The limitation is therefore architecturally bounded rather than baked in.

Chapter 5

System Evaluation

5.1 Evaluation Methodology

The evaluation reported in this chapter assesses the system across two distinct corpora using the from-scratch RAGAS-style answer evaluator described in Section 4.7. The two-corpus design is deliberate: a single evaluation set risks conflating system properties with corpus properties, whereas comparable runs over different document types isolate which behaviours are intrinsic to the architecture and which depend on the structure of the ingested material.

The first corpus (NLP-Papers) comprises twelve seminal machine-learning publications — the original Transformer, BERT, DPR, RAG, HyDE, RAGAS, the Gao et al. RAG survey, ResNet, VGG, GPT-3, InstructGPT (RLHF), and Chinchilla papers — ingested as PDFs. The second corpus (ML-Notes) comprises lecture slides and accompanying notes from a third-year machine-learning module, ingested in mixed PDF and text formats. The two corpora differ markedly in structure: NLP-Papers contains dense academic prose with numbered sections and long paragraphs, while ML-Notes consists of short bullet-pointed slide content with frequent fragmentation across pages. Both corpora were chunked with the system's default `sentence` strategy: sentence-boundary chunking is fast, deterministic, requires no GPU-backed sentence-transformer model at ingestion time, and produces good results for both well-punctuated academic prose and bullet-style slide text. The other five strategies (Table 4.1) are exercised in unit tests and were available at runtime; a controlled ablation comparing chunking strategies on the same evaluation corpora is identified as future work in Section 5.9.

Each corpus was paired with a hand-labelled question set in which the ground-truth answer was authored from the source documents themselves — twelve questions for NLP-Papers and eight for ML-Notes, twenty in total. Test set sizes at this scale are intended to demonstrate behaviour across realistic question types rather

Table 5.1: Answer-quality metrics across the two evaluation corpora. NLP-Papers: $n = 12$; ML-Notes: $n = 8$; combined $n = 20$.

Corpus	**Faith.**	**Relev.**	**Ctx. Prec.**	**RAGAS**
NLP-Papers ($n = 12$)	0.941	0.701	0.823	0.800
ML-Notes ($n = 8$)	0.925	0.798	0.731	0.772
Combined ($n = 20$)	0.934	0.740	0.786	0.788

than to support inferential statistical claims; the limitation is examined explicitly in Section 5.9. All evaluations were performed with Claude as both generator and judge, using identical pipeline configuration — semantic caching disabled to ensure each query is freshly retrieved, top-$k = 8$ retrieval, hybrid BM25–FAISS with RRF and cross-encoder reranking, HyDE expansion enabled, adaptive re-retrieval permitted on grounding scores below 0.5. The full per-query JSON outputs from both runs are included in the project repository under `data/eval/`.

5.2 Headline Results

Table 5.1 summarises the four reference-free metrics across both corpora and aggregated. The composite RAGAS score is the harmonic mean of the three component metrics, computed per query and averaged.

Three observations dominate. First, faithfulness was high in both corpora measured — 0.941 and 0.925 respectively, with the combined mean 0.934. Of the 313 atomic claims extracted across the twenty answers (161 from NLP-Papers, 152 from ML-Notes), the judge LLM rated the great majority as supported by the retrieved context. Hallucination, the principal failure mode RAG is deployed to mitigate, was controlled in this evaluation: the LLM judge marked few atomic claims as unsupported by retrieved context. This figure should be read alongside the same-family judge bias documented in Section 5.8 and the human calibration check reported in Section 5.7: on a ten-question subset re-rated manually, the LLM judge agreed with the human rater on 97% of atomic claims, with the few disagreements running counter to the direction the literature predicts (the judge was stricter than the human, not more lenient). The headline figure is therefore not visibly inflated by same-family bias on the sample examined, though the calibration itself is constrained as documented in Section 5.7. Second, context precision varies meaningfully with corpus structure: NLP-Papers achieves 0.823 while ML-Notes drops to 0.731. The hybrid retrieval pipeline locates relevant chunks more reliably in well-structured academic prose than in fragmented slide-format material — a finding consistent with the chunking literature surveyed in Section 3.8. Third, the

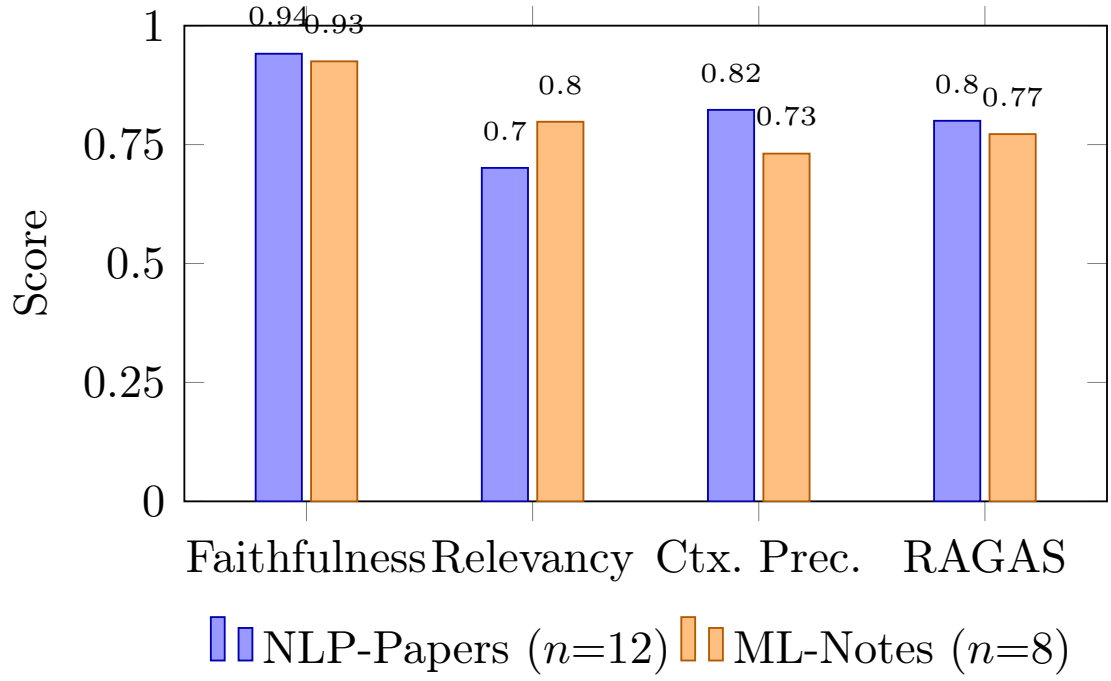

Figure 5.1: Per-metric performance across the two evaluation corpora. Faithfulness varied little between corpora; the other three metrics varied more with document structure.

composite RAGAS score sits at 0.788 overall, with neither corpus dropping below 0.77.

Figure 5.1 visualises the per-corpus breakdown. The structurally different corpora produced different behaviour on every metric except faithfulness; the grounding scorer and fact verifier were designed to protect against the kind of variance that would arise if the generator were free to draw on parametric knowledge, and the observed stability of faithfulness across the two corpora is consistent with that design intent (subject to the validity threats noted in Section 5.8). The headline numbers measure the full pipeline in isolation; Section 5.3 reports a comparison against a stripped-down BM25-only baseline that bounds the contribution of the architectural components themselves.

5.3 Baseline Comparison

The headline figures in Table 5.1 measure how well the full pipeline performs but say nothing about which architectural components contribute the gain. To bound

Table 5.2: Naive BM25-only baseline against the full pipeline on the NLP-Papers corpus (n=12). The same generator, knowledge base, chunker, prompt template, and test set are used for both runs; only the retrieval-and-grounding apparatus differs.

Pipeline	**Faith.**	**Relev.**	**Ctx. Prec.**	**RAGAS**
Full pipeline	0.941	0.701	0.823	0.800
Naive BM25 baseline	0.931	0.777	0.605	0.695
Δ (full − naive)	+0.010	−0.076	+0.218	+0.105

that question empirically, a naive baseline was constructed and run against the same NLP-Papers test set used for the main evaluation.

The baseline retains the knowledge base, the chunker, the test set, and the generator (Claude); it removes everything else. Specifically: BM25 lexical retrieval at k=8 replaces the hybrid BM25–FAISS fusion via Reciprocal Rank Fusion; cross-encoder reranking is disabled; query expansion and HyDE are disabled; the semantic cache is disabled; sentence-level grounding scoring, adaptive re-retrieval, and post-generation fact verification are all bypassed. The retrieved chunks are concatenated and supplied to the generator with the same prompt template the full pipeline uses. The baseline is therefore the simplest pipeline that can plausibly be called RAG — the configuration the architecture is competing against.

Result Table 5.2 reports the comparison. Three observations dominate. First, faithfulness is essentially unchanged (0.941 full vs 0.931 naive); both pipelines feed retrieved chunks to the same generator, and faithfulness behaves as a property of the generator preserved across configurations rather than a property the architecture creates. Second, context precision differs substantially (0.823 full vs 0.605 naive — a 22-point gap); the pre-retrieval components (query expansion, HyDE) and post-retrieval components (RRF, cross-encoder reranking) target retrieval quality directly, and this gap is where the architecture earns its keep. Third, the composite RAGAS gap of approximately 10.5 points is dominated by the context-precision difference. Answer relevancy ran in the opposite direction (0.701 full vs 0.777 naive); the full pipeline produces longer, more comprehensive answers from richer retrieved context, and the reverse-question relevancy metric scores marginally lower on these than on the shorter, more directly question-focused answers the naive baseline produces from sparser context. This is reported here rather than rationalised away, and is flagged as an open finding in Section 5.9.

Qualitative case The starkest single demonstration is NLP-Papers Q9 ("What is the key architectural choice of the VGG network?"). The full pipeline retrieves the original VGG paper at context precision 0.93 and produces a complete answer covering the small 3×3 filters, the depth-through-stacking argument, and the effective receptive-field reasoning. The naive baseline retrieves unrelated material at context precision 0.25, and the generator responds with "I cannot find specific information about the key architectural choice of the VGG network." Same knowledge base, same chunker, same generator, same question, same prompt; only the retrieval pipeline differs. The hybrid retrieval and reranking located the correct paper; BM25 alone did not.

Scope The baseline was run on the NLP-Papers corpus only; the corresponding ML-Notes baseline was not produced and remains as future work (Section 6.3). The single-corpus comparison is sufficient to demonstrate that the architecture's pre-retrieval and post-retrieval components contribute substantially to context precision on the academic-prose corpus, but it does not establish whether the gap generalises to structurally different corpora. The CNN graceful-failure case in Section 5.5 suggests the full pipeline retains its negative-rejection capability on ML-Notes, but this is observation rather than measurement.

5.4 Runtime Performance

The brief asks for performance benchmarks where applicable. The following figures are wall-clock measurements taken from the system's own request-timing middleware during normal operation against an active knowledge base of 3,330 chunks on a commodity Windows workstation; they are intended as orders-of-magnitude indicators rather than as benchmark-grade measurements under controlled load. Table 5.3 summarises the principal endpoints.

Three observations follow. First, retrieval-only operations are fast: hybrid BM25–FAISS search with cross-encoder reranking completes in roughly 110 ms once the cross-encoder is warm, and lightweight metadata endpoints for analytics, history, and annotations return in single-digit milliseconds. The retrieval architecture is not a bottleneck. Second, generation-bound operations are dominated by LLM latency: a full RAG chat query takes approximately 10–15 s once warm, of which the substantial majority is Claude generation time rather than retrieval. Cold-start adds approximately 20 s for one-time loading of the cross-encoder model and Claude SDK initialisation, but this cost is paid once per server lifetime. Third, intelligence-layer operations scale with the number of LLM calls they make: conflict detection at ≈ 20 s involves six pairwise comparisons; quiz generation at ≈ 41 s produces ten multiple-choice and five short-answer questions in a single

Table 5.3: End-to-end latency for representative operations against an active knowledge base of 3,330 chunks. Cold values include one-time model loading; warm values are steady-state.

Operation	**Cold**	**Warm**
Semantic search (no LLM)	—	≈110 ms
Lightweight metadata endpoints	—	2–13 ms
Full chat query (RAG pipeline)	≈32 s	10–15 s
Knowledge-graph load	≈5.5 s	≈3 s
Conflict detection (6 pairs)	—	≈20 s
Study-path generation	—	≈35 s
Quiz generation (15 questions)	—	≈41 s
Document summarisation (95 chunks)	—	≈89 s

Table 5.4: Highest- and lowest-scoring queries per corpus, with the component metric most responsible for the score.

Corpus	**Query**	**RAGAS**	**Driver**
NLP-Papers	"Three RAG paradigms"	0.917	Ctx. Prec. 1.00
NLP-Papers	"What is RLHF"	0.643	Relev. 0.45
ML-Notes	"What is machine learning"	0.901	Ctx. Prec. 0.97
ML-Notes	"What is a CNN"	0.404	Ctx. Prec. 0.20

structured-output call followed by validation; document summarisation at ≈89 s reflects per-chunk synthesis over a 95-chunk academic paper. The latencies are consistent with the architectural choice of offloading reasoning to a high-quality cloud LLM rather than running a smaller local model: the system trades wall-clock time for output quality, an acceptable trade-off for a study-and-research assistant where the user is reading the result rather than waiting on a tight feedback loop.

5.5 Qualitative Analysis: Best and Worst Cases

The aggregate metrics conceal two qualitatively different failure modes that are visible only at the per-question level. Table 5.4 summarises the highest- and lowest-scoring queries in each corpus.

The two best-performing queries share a common shape: each asks for a specific, well-defined concept that appears prominently in the corpus. The Gao et al. RAG survey explicitly enumerates three paradigms in a single subsection, so re-

trieval finds adjacent and highly relevant chunks; context precision reaches the theoretical maximum of 1.00, and the answer follows the source structure faithfully. The "what is machine learning" query benefits similarly from the introductory lecture slide that defines the term explicitly.

The two worst-performing queries are pedagogically more interesting because they fail in qualitatively different ways. The RLHF query returns a faithfulness of 1.00 but an answer relevancy of only 0.45. Inspecting the answer reveals the cause: the generator produced an extremely thorough response covering the historical context, implementation pipeline, safety measures, and architectural limitations of RLHF. None of this content was hallucinated — every claim is correctly entailed by the InstructGPT paper — but the breadth of the answer caused the LLM-generated reverse-questions to diverge from the original specific question, depressing the cosine-similarity component of relevancy. This is a verbosity failure, not a correctness failure, and points toward future work on response-length calibration rather than to any defect in the retrieval or grounding pipeline.

The CNN query is the more architecturally significant case. Its RAGAS score of 0.404 initially reads as a clear failure, but the component breakdown reverses that interpretation: faithfulness is 1.00, answer relevancy is 0.75, and the score is dragged down almost entirely by context precision of 0.20. The reason is that the ML-Notes corpus contains no dedicated CNN material — the lecturer's notes only mention CNNs in passing as a future-project component. Rather than invent CNN content from general neural-network material, the system's answer states explicitly that "the context doesn't contain enough specific information about Convolutional Neural Networks" and offers to address the question if additional documents were ingested. This is precisely the negative-rejection behaviour that Chen et al. identify as a fundamental capability requirement for production RAG [7]: the system must refuse to answer questions for which it lacks evidence rather than generate plausible-sounding fabrications. The architecture's grounding scorer and fact verifier (Section 4.5) are responsible for this behaviour; the low context precision score, far from being a problem, is the metric correctly registering that retrieval had little to return. A pure quality-of-output assessment would rate this query a success.

5.6 Mapping Results to Project Objectives

The brief requires that evaluation address each of the objectives stated in the introduction. The mapping is summarised below.

O1: Multi-format ingestion. All four file formats present in the two evaluation corpora — PDF, DOCX, Markdown, plain text — ingested without manual intervention. The change-detection mechanism in `DocumentRegistry` successfully

short-circuited re-ingestion on unchanged files in repeated test runs.

O2: Six chunking strategies. The default `sentence` strategy was used for both evaluation corpora; the remaining five strategies were exercised in unit tests covering chunk-size distribution, boundary detection, and metadata preservation across all 522 tests. A controlled head-to-head retrieval comparison *across* all six strategies on identical corpora — enabled by `ChunkManager.compare_strategies()` — was implemented but not run as part of the formal evaluation, and remains a natural extension.

O3: Hybrid retrieval. The retrieval components are verified in two ways. Directly, the unit-test suite covers RRF score accumulation, cross-encoder reranking, the three query-expansion strategies, and entity-aware boosting. Indirectly, the context precision of 0.786 across the combined test set indicates that the BM25–FAISS–RRF–cross-encoder pipeline locates relevant material more often than not, since context precision is a direct function of retrieval order. The reliance on indirect evidence is partly principled: Salemi and Zamani [31] report that traditional document-level IR metrics correlate only weakly with downstream RAG answer quality, suggesting that answer-level context-precision is a more relevant signal of retrieval quality for RAG specifically than chunk-level Precision@K would be. Nevertheless, producing a chunk-level labelled ground-truth set and reporting Precision@K, MRR, and nDCG@K from `src.evaluation.metrics` alongside the answer-level metrics remains a valuable extension and is identified as future work in Section 5.9.

O4: Full RAG pipeline. Faithfulness of 0.934 aggregated demonstrates that the eleven-stage pipeline including grounding-driven adaptive retry produces answers reliably entailed by retrieved content. The CNN case in Section 5.5 confirms that graceful degradation under retrieval failure works as designed.

O5: Two evaluation layers. The classical IR layer (`src.evaluation.metrics`) is exercised by the unit-test suite and used during development for retrieval-regression checks. The RAGAS-style answer evaluator (`src.evaluation.answer_eval`) is the source of every quantitative result reported in this chapter — the layer is not only implemented but actively producing the evidence on which the system is being judged.

O6: Three interfaces and a verified test suite. CLI, Python API, and Flask web application all share the same backend; the test suite of 522 tests across 29 modules passes as a precondition for each evaluation run.

All six objectives are addressed by the evidence presented in this chapter or by the unit-test suite supporting it.

5.7 Human Calibration of the LLM Judge

The same-family judge bias (Section 5.8 below) is the single largest threat to the absolute magnitudes reported in Table 5.1: an evaluator drawn from the same model family as the generator is theoretically prone to overestimate faithfulness. To bound the size of any such inflation in this evaluation, a ten-question subset was re-rated manually.

The picks were deterministic to forestall selection bias: for each corpus, every other question up to a cap of five was selected. This yielded NLP-Papers questions 1, 3, 5, 7, 9 and ML-Notes questions 1, 3, 5, 7, 8. The picked subset spans the full range of LLM-judge faithfulness scores across the corpora (0.778 to 1.000) and includes the CNN failure case discussed in Section 5.5.

For each picked question the answer, the retrieved context, and the LLM judge's atomic-claim decomposition were re-examined. Each atomic claim was rated as either supported or unsupported on the same binary criterion the LLM judge uses — whether a paraphrase of the claim appears in the retrieved chunks. The retrieved chunks were re-fetched from the system rather than reconstructed from the truncated previews stored in the JSON outputs, ensuring the human rater saw what the judge saw.

Result Across 166 atomic claims spanning the ten questions, the human rater agreed with the LLM judge on 161 (97.0%). All five disagreements went in the same direction: the LLM judge had marked the claim unsupported and the human marked it supported. There were no disagreements in the opposite direction. The disagreements concentrated on two questions: NLP-Papers Q5 (Gao et al. paradigms, where the judge marked "the survey identifies three paradigms" as unsupported despite the cited paper using the exact phrase) and ML-Notes Q7 (gradient descent, where the judge rejected four general-knowledge statements consistent with but not verbatim in the retrieved chunks). Per-question mean faithfulness across the ten-question subset rose from 0.922 (LLM judge) to 0.951 (human rater) — a difference of approximately three percentage points.

Interpretation The direction of this disagreement is the opposite of what the same-family bias literature predicts. Rather than the judge overestimating faithfulness in the generator's favour, the LLM judge in this evaluation is, on the calibration subset, slightly stricter than a careful human reader. The implication is that the absolute magnitude of 0.934 reported in Table 5.1 is not visibly inflated relative to a human reader on this sample, and may be a slight underestimate where answers contain general-knowledge statements consistent with but not verbatim in the retrieved chunks.

This finding bounds rather than removes the same-family bias concern. The calibration is constrained in three ways. First, the human rater is the system's developer; an author-as-evaluator bias toward generosity is plausible even in deliberate ratings. Second, the sample is ten questions and one rater, which is too small for inferential statistical claims about agreement. Third, a stricter test would be a blinded multi-rater calibration with raters who have no stake in the system's reported performance. None of these constraints is mitigated by the present check. They are recorded explicitly here to prevent the calibration from being read as stronger evidence than it is. A larger blind multi-rater calibration remains as future work.

5.8 Threats to Validity

Several factors could distort the magnitudes reported in Table 5.1 in ways that the limitations subsection below does not address. They are recorded explicitly here because honest evaluation requires distinguishing between gaps in coverage and biases in the existing results.

Author-as-evaluator The test questions and ground-truth answers were authored by the system's developer, who has prior knowledge of which retrieval patterns succeed on each corpus. There is no protection against the unintentional authoring of questions that the system happens to handle well; a third-party question set produced without sight of the system's behaviour would constitute stronger evidence.

Compounding judge–corpus alignment Both the ground-truth answers and the candidate answers are written from the same source documents, and the LLM-as-judge sees both alongside the retrieved context. Any answer that paraphrases the ground-truth will trivially appear faithful, regardless of whether the underlying retrieval pipeline produced the supporting evidence or whether the generator relied on parametric knowledge. This biases faithfulness upward by an amount the present evaluation cannot quantify.

Small-n sensitivity With twenty labelled queries across the two corpora, a single edge-case query can move aggregate metrics by approximately five percentage points. The CNN failure case in Section 5.5 alone reduces the ML-Notes corpus mean RAGAS score by roughly 0.05. Aggregate results should therefore be read as approximate rather than precise.

Reranker domain mismatch The cross-encoder reranker (`ms-marco-MiniLM-L-12-v2`) was trained on a web-search relevance task. Whether its scoring transfers cleanly to academic prose and lecture-slide content was not measured directly; the reported context-precision figures absorb any such mismatch as part of the headline result.

These threats do not invalidate the principal finding, but should qualify how confidently the absolute numbers can be cited.

5.9 Limitations

Several limitations of the evaluation warrant explicit acknowledgement.

Test-set size Twenty labelled queries across two corpora is sufficient to demonstrate behaviour and to surface qualitatively distinct failure modes, but is too small to support claims of statistical significance. A test set of one hundred queries per corpus would permit confidence intervals on each metric and a meaningful power analysis; the labour cost of producing such a set under hand-authored ground-truth answers was beyond the scope of this project.

Same-family judge As acknowledged in Section 3.11 and Section 4.10, both the answer generator and the LLM-as-judge for the RAGAS metrics are Claude. This introduces same-family bias of the kind documented by Zheng et al. [28]: the judge may be primed to accept outputs in its own stylistic register. The bounded human calibration in Section 5.7 produces some reassurance — on the ten-question subset re-rated, the LLM judge agreed with the human rater on 97% of atomic claims and the disagreements ran counter to the predicted bias direction — but the calibration itself is constrained by being a single-rater author check. The architecture mitigates the bias structurally through the `LLMProvider` abstraction; a future evaluation pass with a different judge model would require only a configuration change.

Single-run results The reverse-question generation step of the answer-relevancy metric uses a non-zero generation temperature, making the metric weakly non-deterministic between runs. Each query in this evaluation was scored once. A repeated-trials design with ten or more independent runs per query would yield per-metric standard deviations and tighter confidence intervals; this was not pursued at the present scale because the variance contribution from temperature 0.3 on a three-question reverse-question set is small relative to the inter-query variation visible in Table 5.4.

Single-corpus baseline comparison The naive baseline reported in Section 5.3 was run on the NLP-Papers corpus only. The corresponding ML-Notes baseline was not produced, so the architecture's value-add on bullet-style lecture-slide content is bounded only by the qualitative behaviour observed in Section 5.5 (graceful failure in the CNN case) rather than by direct measurement against a stripped-down pipeline. Whether the 22-point context-precision gap demonstrated on academic prose generalises to structurally different corpora is therefore an open empirical question. Symmetric ML-Notes baseline results are the obvious next experiment.

Relevancy reversal The naive baseline scored higher than the full pipeline on answer relevancy (0.777 vs 0.701). The most plausible mechanism is that the full pipeline retrieves richer context and the generator produces longer, more comprehensive answers, which the reverse-question relevancy metric scores marginally lower than shorter, more directly question-focused answers from sparser context. This is consistent with how the metric is constructed (cosine similarity between the original question and reverse-questions generated from the answer alone) but warrants further investigation. A targeted experiment varying answer length while holding retrieval quality constant would isolate whether the metric is penalising answer comprehensiveness or reflecting a real relevancy reduction.

Closed-domain validation Both evaluation corpora come from academic and educational sources; the system has not been evaluated on legal, medical, or commercial documents where vocabulary, structural conventions, and the consequences of error all differ. The architecture is not domain-coupled and should transfer, but transfer is not demonstrated by the present evaluation. Barnett et al. document how RAG failure modes vary substantially across deployment domains in their experience report from research, education, and biomedical case studies [32]; the absence of cross-domain validation is therefore a genuine limitation rather than a presentational one.

Recommended extensions Three concrete additions would strengthen the evaluation. First, producing chunk-level relevance labels would allow the classical IR metrics (Precision@K, MRR, nDCG@K) to be reported alongside the answer-level metrics, providing direct rather than indirect retrieval-quality evidence. Second, a chunker ablation study using `ChunkManager.compare_strategies()` on identical corpora would quantify the impact of chunking choice on downstream answer quality. Third, repeated trials with a different judge LLM would address both the nondeterminism and same-family-bias limitations in a single pass. None of these is technically blocked by the architecture; each is a matter of additional

labelled data or compute time.

These limitations bound the conclusions that can be drawn. With that caveat in mind, the evaluation produces concrete observations: across two structurally distinct corpora and twenty labelled queries, the system maintained faithfulness above 0.92 and a composite RAGAS score above 0.77 on each corpus, and the one tested degeneracy case showed graceful failure rather than fabrication. These results are consistent with the central claim of this dissertation — that careful integration of established retrieval and generation techniques, instrumented with reference-free evaluation, can produce a Personal Knowledge Assistant suitable for document-grounded question answering, subject to the validity threats catalogued above and the calibration work flagged as future work.

Chapter 6

Conclusion

6.1 Outcomes

The Personal Knowledge Assistant has been designed, implemented, and evaluated as a complete end-to-end Retrieval-Augmented Generation system for document-grounded question answering. The principal outcome of the project is a working system that meets each of the six objectives stated in Section 1.4.

O1 (multi-format ingestion) is met by the `DocumentManager` factory and three concrete parsers covering PDF, DOCX, Markdown, and plain text, paired with a SHA-256 document registry that short-circuits re-ingestion of unchanged files. **O2 (multiple chunking strategies)** is met by six interchangeable chunkers — fixed, sentence, embedding-similarity, density-clustering, topic-modelling, and recursive-hierarchical — exposed through the `ChunkManager` configuration interface. **O3 (hybrid retrieval)** is met by `HybridSearcher`, combining BM25 and FAISS through Reciprocal Rank Fusion with cross-encoder reranking and three query-expansion strategies including HyDE. **O4 (full Advanced-RAG pipeline)** is met by `RAGPipeline`, which orchestrates eleven stages including semantic caching, conversational query rewriting, sentence-level grounding, adaptive re-retrieval, and post-generation fact verification, and which preserves the highest-confidence attempt across retries rather than the latest. **O5 (two-layer evaluation)** is met by the parallel `src.evaluation.metrics` and `src.evaluation.answer_eval` modules, the latter of which produced the quantitative results in Chapter 5. **O6 (three interfaces and a verified test suite)** is met by the CLI, Python API, and Flask twelve-view web application, all sharing a common backend, verified by 522 tests across 29 modules — exceeding the target of five hundred tests across twenty-five modules.

6.2 Principal Findings

Three findings from Chapter 5 merit explicit restatement.

The first is that faithfulness varied little between the two corpora measured, despite their substantially different structures. Across dense academic prose and bullet-style lecture slides, the sentence-level grounding scorer and post-generation fact verifier held faithfulness above 0.92, with 313 atomic claims extracted across twenty answers and the great majority rated as supported by retrieved context. This is the property the architecture was designed to protect, and within the bounds of the validity threats catalogued in Section 5.8 the observed behaviour is consistent with that design intent.

The second is that retrieval quality is corpus-dependent in a predictable direction. Context precision was 0.823 on the academic-prose corpus and 0.731 on the lecture-slide corpus; the hybrid retrieval pipeline locates relevant content more reliably in well-structured prose than in fragmented slide content. This is consistent with the chunking literature [25] and points toward structure-aware chunking as the natural extension for slide-format and other non-prose corpora.

The third is that the one tested degeneracy case showed graceful failure rather than fabrication. The CNN query (Section 5.5), with a composite RAGAS score of 0.404, is on inspection a success: the corpus contained no dedicated CNN content, and the system stated as much rather than fabricating an answer. This is precisely the negative-rejection capability that Chen et al. identify as a fundamental requirement for production RAG [7], and the architecture's grounding scorer and fact verifier are responsible for the behaviour observed in this case.

6.3 Future Work

The dissertation identifies several concrete extensions, three methodological and three architectural.

The methodological extensions strengthen the empirical foundation on which the system's claims rest. First, producing chunk-level relevance labels for the existing twenty-question corpus would allow direct retrieval-quality reporting (Precision@K, MRR, nDCG@K) alongside the answer-level metrics, addressing the indirect-evidence weakness in the O3 mapping. Second, a chunker ablation study using `ChunkManager.compare_strategies()` on identical corpora would quantify the impact of chunking choice on downstream answer quality and would justify the architectural decision to implement six strategies rather than one or two. Third, repeated trials with a different judge LLM — a local Llama-class model or a non-Anthropic cloud model — would address the same-family-bias and nondeterminism limitations in a single pass.

The architectural extensions follow from the de-scoping decisions documented in Section 2.7. The clean abstraction at `LLMProvider` admits a local language-model implementation without modifying any other component; the embedding service can be swapped for a fine-tuned variant; LoRA can be wired in beneath the provider abstraction. The original proposal's privacy-first local deployment is therefore reachable as an extension of the current system rather than as a rewrite. A second architectural extension would replace the co-occurrence knowledge graph with learned relation extraction, producing typed edges that distinguish "X criticised Y" from "X agreed with Y" — the principal weakness of the current graph implementation (Section 4.10). A third would extend ingestion to streaming sources — email, RSS feeds, web bookmarks — which the proposal anticipated but the delivered system does not implement.

6.4 Critical Reflection

Four decisions made during development warrant explicit acknowledgement alongside the outcomes claimed above. They are recorded here because the quality of a project is partly judged by the maturity of its self-assessment.

The first concerns scope. Implementing six interchangeable chunking strategies was research-thorough — the catalogue stands as evidence of breadth and supports the comparative analysis flagged as future work — but a leaner system with two strategies (`sentence` for general use, `recursive_hierarchical` for structured documents) would have delivered the same headline evaluation findings with substantially less code to maintain, fewer tests to write, and less ingestion-path complexity. The wider catalogue belongs in a research artefact more than in a production deliverable; a tighter architecture would have been more defensible.

The second concerns timing. The RAGAS-style answer evaluator was built late in the project, after the principal retrieval and generation features were already in place. The classical IR metrics (`src.evaluation.metrics`) supported regression testing throughout, but answer-quality regressions can only be caught once an answer-level evaluator exists. Building even a minimal version of the answer evaluator earlier would have surfaced grounding-scorer issues of the kind described in Section 2.6 during development rather than at evaluation time, and would have shortened the feedback loop on prompt-engineering decisions throughout the RAG pipeline.

The third concerns evaluation discipline. The minimal human spot-check reported in Section 5.7 was added late in the project rather than designed in from the start. Although the result it produces is reassuring — the LLM judge agreed with the author's manual rating on 161 of 166 atomic claims, with the few disagreements running counter to the direction the same-family bias literature predicts —

the calibration is constrained by being a single-rater check by the system's developer rather than a blinded multi-rater calibration with non-author evaluators. A more rigorous calibration with independent raters would be the natural next step.

The fourth concerns comparative measurement. The naive baseline reported in Section 5.3 was constructed late in the project and run only against the NLP-Papers corpus. The result is informative — a 22-point context-precision gap and a 10.5-point RAGAS gap on the corpus measured, with the largest single qualitative demonstration being a question the full pipeline answers correctly while the baseline cannot answer at all. But two methodological improvements would have been preferable. First, the baseline should have been built early and run regression-style throughout development; the architecture's value-add would then have been visible at every milestone rather than only at evaluation time. Second, a symmetric ML-Notes baseline should have been produced; without it, the gap demonstrated on academic prose cannot be generalised to structurally different corpora. Together, the late spot-check and the asymmetric baseline are the most actionable methodological lessons from the evaluation phase.

6.5 Closing Reflection

The dissertation set out to build a Personal Knowledge Assistant that answers questions about a user's own documents with citations and without inventing sources that do not exist. The system as delivered performs this task: across two evaluation corpora and twenty labelled queries, faithfulness exceeded 0.92, the composite RAGAS score sat above 0.77 on each corpus, and the one tested degeneracy case (Section 5.5) showed the architecture's grounding mechanisms producing an acknowledgement of insufficient context rather than a fabrication. The contribution is not novelty in any single technique, all of which are drawn from the established literature, but in their disciplined integration into a coherent, evaluable, testable system, with the evaluation framework instrumented to make the system's behaviour examinable and its limits explicitly catalogued (Section 5.8).

The broader observation is that the techniques required to build such a system are now mature and accessible. BM25 is more than two decades old; sentence-transformer embeddings, FAISS, and cross-encoder reranking have been production-grade for several years; RAG itself is five years old; the RAGAS framework is two years old. What was once a research-frontier capability is now an engineering integration problem. The contribution this work makes is to demonstrate that the integration can be done well: rigorously specified, cleanly architected, properly tested, honestly evaluated.

Bibliography

[1] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.

[2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 9459–9474, 2020.

[6] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997, 2024.

[7] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. In *Proc. AAAI*, pages 17754–17762, 2024.

[8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. arXiv preprint arXiv:2309.15217, 2023.

[9] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proc. EMNLP-IJCNLP*, pages 3982–3992, 2019.

[11] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proc. SIGIR*, pages 758–759, 2009.

[12] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv preprint arXiv:1901.04085, 2019.

[13] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[14] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, and Dave Thomas. Manifesto for agile software development. `https://agilemanifesto.org/`, 2001.

[15] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK, 2008.

[16] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1994.

[17] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. NAACL-HLT*, pages 4171–4186, 2019.

[20] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proc. EMNLP*, pages 6769–6781, 2020.

[21] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. arXiv preprint arXiv:2401.08281, 2024.

[22] Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. In *Journal of Open Source Software*, volume 2, page 205, 2017.

[23] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. arXiv preprint arXiv:1802.03426, 2018.

[24] Maarten Grootendorst. BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv preprint arXiv:2203.05794, 2022.

[25] Antonio Jimeno Yepes, Yao You, Jan Milczek, Sebastian Laverde, and Renyu Li. Financial report chunking for effective retrieval augmented generation. arXiv preprint arXiv:2402.05131, 2024.

[26] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. arXiv preprint arXiv:2212.10496, 2022. Hypothetical Document Embeddings (HyDE).

[27] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *Proc. ICML*, pages 3929–3938, 2020.

[28] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685, 2023.

[29] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy: Industrial-strength natural language processing in Python. *Zenodo*, 2020.

[30] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proc. 7th Python in Science Conf. (SciPy)*, pages 11–15, 2008.

[31] Alireza Salemi and Hamed Zamani. Evaluating retrieval quality in retrieval-augmented generation. In *Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, pages 2395–2400, 2024.

[32] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. In *Proc. IEEE/ACM Int. Conf. AI Engineering – Software Engineering for AI (CAIN)*, pages 194–199, 2024.

Appendix A

Project Repository

The complete source code, test suite, evaluation results, and supporting documentation for the Personal Knowledge Assistant are available at `https://github.com/PatrickClerkin/Personal-Knowledge-Assistant`. Installation instructions, dependency lists, and a quick-start guide are provided in the repository's `README.md`.